

People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

Université Badji Mokhtar –
Annaba
Badji Mokhtar – Annaba
University



Université Badji Mokhtar
– Annaba –

Faculty: Technology
Department: Computer Science
Domain: Mathematics – Computer Science
Field: Computer Science
Speciality: Machine Learning & Intelligent Systems

Memoir

Presented for the award of the **Master's Degree**

Theme

**Leveraging Deep Learning Techniques for Advanced
Recommender Systems**

Presented by: Farouk Ouled Meriem

Supervisor: Samira Lagrini [Grade] Badji Mokhtar – Annaba University

Defense Jury:

[Jury Member 1]	MCB	Badji Mokhtar – Annaba University	President
Samira Lagrini	MCA	Badji Mokhtar – Annaba University	Supervisor
[Jury Member 2]	MCB	Badji Mokhtar – Annaba University	Examiner

Acknowledgements

First of all, I express my deepest gratitude to **Allah** for granting me the strength and perseverance to accomplish this work.

I sincerely thank my **parents** for their unconditional love, support, and sacrifices throughout my studies.

My heartfelt thanks go to my supervisor for their constant guidance, availability, and valuable feedback at every stage of this work.

I also extend my appreciation to the members of the jury for their time and careful evaluation of this memoir.

Finally, I thank all my colleagues and friends for their encouragement and support throughout this journey.

Dedications

*To my parents,
for every sacrifice they made so I could reach this day.*

*To my family and friends,
for their constant support and encouragement.*

Farouk Ouled Meriem

Contents

Acknowledgements	i
Dedications	ii
General Introduction	1
1 Foundations and State of the Art	4
1.1 Introduction to Recommendation Systems	4
1.2 Sequential Recommendation	5
1.3 Sequence-Based Models	6
1.3.1 GRU4Rec	7
1.3.2 Caser	7
1.3.3 SASRec	7
1.3.4 BERT4Rec	7
1.4 Graph-Based Models / GNN	8
1.4.1 LightGCN	9
1.5 LLMs Applied to Sequential Recommendation	10
1.6 Evaluation Metrics	11
1.6.1 Hit Ratio at K	11
1.6.2 Normalized Discounted Cumulative Gain at K	12
1.6.3 Mean Reciprocal Rank	12
1.7 Conclusion	13
1.8 Identified Gaps and Positioning of Contributions	13
2 Length-Adaptive Hybrid GNN-Transformer for Sequential Recommendation	14
2.1 Introduction	14
2.2 Limitations of Existing Models and Problem Statement	14
2.3 Proposed Solution: The Adaptive Fusion Mechanism	17
2.3.1 Formal Problem Definition	17
2.3.2 The Adaptive Fusion Mechanism	17
2.3.3 Intuition and Rationale	18
2.4 Family of Proposed Hybrid Models	18
2.4.1 Hybrid BERT4Rec + LightGCN with Fixed Fusion	18
2.4.2 Hybrid BERT4Rec + LightGCN with Discrete Fusion	18

2.4.3	Hybrid BERT4Rec + Temporal Graph Transformer (TGT) . . .	19
2.4.4	Hybrid BERT4Rec + Temporal Convolutional Network (TCN) .	19
2.5	Conclusion	20
3	Experiments and Results: Length-Adaptive Hybrid GNN-Transformer	21
3.1	Introduction	21
3.2	Datasets	21
3.3	Preprocessing	22
3.4	Hyperparameters	24
3.5	Baselines	25
3.6	Results	28
3.7	Ablation Study	38
3.8	Conclusion	41
4	Item Description Strategies for LLM-Based Sequential Recommendation	42
4.1	Introduction	42
4.2	The LLM2Sequential Pipeline	42
4.3	Item Description Strategies	43
4.4	Experimental Setup	45
4.5	Conclusion	45
5	Experiments and Results: PromptCraft-SeqRec	47
5.1	Introduction	47
5.2	Recommendation Results	47
5.3	Embedding Quality Analysis	50
5.4	Discussion	51
5.5	Conclusion	52
	General Conclusion	54
	References	56

List of Figures

1.1	Taxonomy of Recommendation System Types.	5
1.2	Illustration of a user interaction timeline for next-item prediction. . . .	6
1.3	Architecture illustration of a sequence-based model such as SASRec or BERT4Rec.	8
1.4	Illustration of a user-item bipartite graph and embedding propagation in LightGCN.	9
1.5	Pipeline for using item descriptions to generate embeddings with an LLM and feed them into a recommender.	10
2.1	Comparison of sparse and dense user history profiles and their ideal information sources.	16
3.1	Preprocessing pipeline applied to both datasets.	23
3.2	Test metrics heatmap on ML-1M. Darker cells indicate higher values. Hybrids marked with †.	29
3.3	Test metrics heatmap on ML-100K. Darker cells indicate higher values. Hybrids marked with †.	29
3.4	Training loss per epoch on ML-1M for all models.	30
3.5	Validation NDCG@10 per epoch on ML-1M. Filled circles mark best checkpoint per model.	31
3.6	Training loss per epoch on ML-100K for all models.	31
3.7	Validation NDCG@10 per epoch on ML-100K. Filled circles mark best checkpoint per model.	32
3.8	HR@10 broken down by user history length group on ML-1M.	34
3.9	HR@20 broken down by user history length group on ML-1M.	34
3.10	NDCG@10 broken down by user history length group on ML-1M. . . .	35
3.11	NDCG@20 broken down by user history length group on ML-1M. . . .	36
3.12	MRR@10 broken down by user history length group on ML-1M. . . .	36
3.13	MRR@20 broken down by user history length group on ML-1M. . . .	37
3.14	HR@10 broken down by user history length group on ML-100K. . . .	38

List of Tables

1.1	Representative sequence-based recommendation models.	8
1.2	Representative graph-based recommendation models.	9
1.3	Examples of how LLMs are used in sequential recommendation.	11
1.4	Ranking Metrics for Sequential Recommendation.	12
2.1	Overview of the four proposed hybrid model variants.	19
3.1	Statistics of the datasets used in this chapter after preprocessing.	22
3.2	Hyperparameter settings for all models.	26
3.3	Summary of baseline models used in this chapter.	27
3.4	Test performance on MovieLens-1M. Best results bold, second-best underlined. † = proposed hybrid.	28
3.5	Test performance on MovieLens-100K. Best results bold, second-best underlined. † = proposed hybrid.	28
3.6	Per-bucket HR@10 / NDCG@10 / MRR@10 on ML-1M and ML-100K. † = proposed hybrid.	33
3.7	Ablation study on ML-1M (HR@10 / NDCG@10 / MRR@10).	39
3.8	Model efficiency and statistical significance on ML-1M. † = proposed hybrid. * $p < 0.05$	40
4.1	The seven item description strategies with illustrative examples. The example item is a moisturizing cream from the Amazon Beauty dataset.	46
5.1	Recommendation results on Amazon Beauty for all description strategies. SASRec hyperparameters are held constant across all runs. $\Delta\text{NDCG@10}$ is relative to the T1 baseline.	48
5.2	Intrinsic embedding quality metrics for each description strategy.	50

General Introduction

Context and Motivation

The era of digital transformation witnesses millions of users being connected via networks and online platforms, serving as essential components of people’s everyday lives. Various websites including E-commerce platforms have allowed people to purchase/sell products worldwide, whereas online platforms like Youtube facilitate instant access to numerous kinds of entertainment such as movies, music and shows. Because of the rapid developments that happened in the digital domain, the amount of contents created online had a tremendously exponential growth. Such vast amount of information available made people very difficult to make a proper choice or obtain relevant content corresponding to user interest. Under such overwhelming amount of choices available, user’s decision making might be impaired or confused. The emergence of recommendation systems is to tackle with this kind of problems. By learning the user interests and behavior, recommendation systems can facilitate users discover desired products, services or content. Recommendation systems contain several categories and sequential recommendation is one of the important ones and has been considered a major research direction in recent years. Unlike traditional collaborative filtering approaches, which identify user interests by group similarities, sequential recommendation models usually take advantage of user interaction history where temporal order matters, so as to predict the next item the user will likely interact with - such as purchase, watch or listen to. By mining the user’s past sequence of behaviors, this kind of model has better performance in capturing dynamic user interests and therefore better recommends relevant contents. The booming of deep learning has been a driving force in the sequential recommendation area in recent years. Early work based on Recurrent Neural Networks, such as [Hidasi et al., \(2016\)](#), is specifically designed for sequences where order is important. These models capture short-term temporal dynamics of user behavior. Attention mechanisms and Transformer-based architectures provided a great breakthrough. Models like SASRec ([Kang and McAuley, 2018](#)) and BERT4Rec ([Sun et al., 2019](#)) capture long-range dependencies within user interaction sequences, and both show better accuracy and scalability than previous work. Meanwhile, Graph Neural Networks were introduced to model the global relation of users and items over the interaction graph and to obtain rich collaborative information that traditional sequential models could not discover. However, these two types of methods have their respective strengths and weaknesses. Graph Neural Networks excel in capturing the

global co-occurrence relations in the interaction graph but are weaker in modelling the local sequential context of users. Sequence-based Transformer models are strong in modelling a user’s recent context information and preferences, but fail to capture global information. The current works generally combine these two paradigms in a fixed way with fixed contribution weights. However, it is obvious that users do not have the same history interaction density; thus, the contribution of these two sources of information should be balanced according to the interaction density of the user history. Users with sparse history interactions need to rely more on the global collaboration signal provided by GNN-based models, while for users with rich interaction histories, Transformer-based models are capable of exploiting detailed sequential information. Simultaneously, Large Language Models (LLMs) such as GPT, LLaMA, and BERT, which are pre-trained on large text corpora, show an extraordinary ability to capture semantic knowledge about the world. As a result, the boom of LLMs has greatly propelled a new revolution in recommendation systems through their deep semantic understanding of user interests, items, and context. However, a fundamental problem remains poorly explored: how to effectively represent the textual descriptions of items so as to maximally leverage the semantic information contained in LLM embeddings for sequential recommendation tasks? In fact, different representation approaches lead to intrinsically different viewpoints of an item, which may significantly affect the quality and distinctiveness of the resulting embeddings.

Addressed Research Problems

This memoir addresses two complementary research problems:

Problematic 1 - *How to dynamically adapt the fusion between global graph signals and local sequential modelling according to each user’s interaction history profile?*

Problematic 2 - *How does the choice of item textual description strategy influence the quality of LLM embeddings in sequential recommendation?*

Main Contributions

This work presents two main contributions:

- **Contribution 1:** A hybrid GNN-Transformer model with adaptive fusion based on user interaction history length.
- **Contribution 2:** A systematic study of the impact of item description strategies on LLM embedding quality for sequential recommendation.

Memoir Organization

The rest of this memoir is structured as follows:

Chapter 1 describes the basic concepts and the state-of-the-art methods for sequential recommendation systems. It covers classical approaches, sequence-based and graph-based deep learning models, Large Language Models, and evaluation criteria. It ends with the research gaps that motivated this work.

Chapter 2 proposes our first contribution: a Length-Adaptive Hybrid GNN-Transformer. It describes the shortcomings of previous models, formalizes the problem, and provides the details of the proposed architecture and fusion method.

Chapter 3 illustrates the experiments of the first contribution. It contains the description of datasets, pre-processing, baselines, parameters, experimental results, and an ablation study.

Chapter 4 discusses the second contribution: a comprehensive analysis of item description strategies for LLM-based sequential recommendation. It presents the **LLM2Sequential** pipeline and seven description strategies proposed in this work.

Chapter 5 presents the evaluation experiments of the second contribution and compares all the strategies against a previous work ([Boz et al., 2025](#)).

Lastly, a **General Conclusion** presents the main findings, the limitations, and future research directions.

Foundations and State of the Art

1.1 Introduction to Recommendation Systems

Recommendation systems (RS) are systems and methods for recommending to users interested items, by filtering large amounts of data according to their tastes and interests ([Castells and Jannach, 2023](#)). As information grows exponentially - from the product catalog on an e-commerce store to the TV shows and movies on Netflix - users can no longer access and process all available information, and the information overload problem makes it increasingly hard for users to find items and pieces of information of interest. The goal of RS is to identify item patterns according to the characteristics of items and the interest profile of the user, and recommend items by analyzing behavioral characteristics. Recommendation systems are essential components of modern intelligent applications such as Netflix, Amazon, and Spotify ([Raza et al., 2024](#)).

There are three common paradigms used to develop a recommendation system, as illustrated in Figure 1.1:

- **Collaborative Filtering (CF):** Recommends items by leveraging the past behavior of users, on the assumption that users who agreed in the past will continue to agree in the future. CF can either use the interaction data itself (memory-based) or latent factor models learnt from the data (model-based) ([Castells and Jannach, 2023](#)).
- **Content-Based Filtering (CB):** Recommends items with the same characteristics as the ones the user previously showed affinity for, by comparing item features with an individual user profile ([Raza et al., 2024](#)).
- **Hybrid Systems:** Combinations of CF and CB methods are often developed in order to overcome the drawbacks of both methods - for example cold-start in CF and over-specialization in CB - and therefore build more efficient recommendation systems ([Raza et al., 2024](#)).

Out of the above, sequential recommendation systems form a subcategory and have gradually become a more and more important area of collaborative filtering, where the order in which users interacted with items is considered. This work concentrates only on this type of systems. Formally they are defined below.

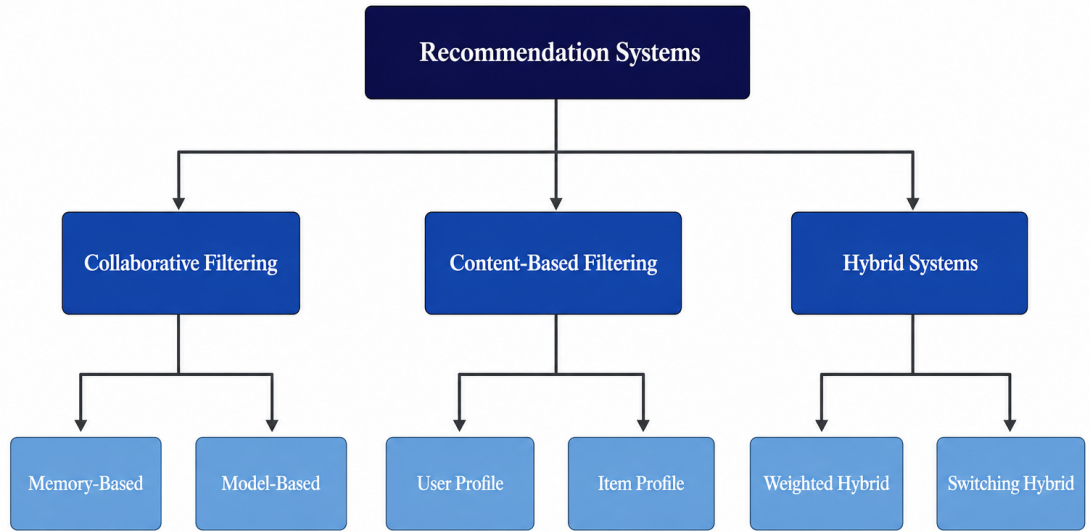


Figure 1.1: Taxonomy of Recommendation System Types.

1.2 Sequential Recommendation

In sequential recommendation (SR), the task of predicting the next item to recommend to a user depends not only on the set of items with which the user has interacted, but also on the specific order of these interactions (Pan et al., 2025). This sequence is important, as users’ preferences are dynamic: short-term preferences often have higher weight than long-term ones and the meaning of the action is interpreted differently based on the context that precedes it (for example, clicking an item does not carry the same semantics as purchasing it). SR is typically framed as the task of predicting the next item given a sequence of past interactions and is naturally suited for scenarios where dynamic, short-term intent needs to be modeled, such as e-commerce, news recommendation, or media streaming (Hidasi et al., 2016; Kang and McAuley, 2018; Sun et al., 2019).

Deep learning has led to major advances in the area of SR, due to models that are able to incorporate sequential dependencies explicitly. Among the early ones, the use of gated recurrent networks such as GRU4Rec was shown to be effective in capturing session-level dynamics and improving next-item prediction (Hidasi et al., 2016). Self-attention based models, like SASRec, proposed an effective solution to capture long-range dependencies (Kang and McAuley, 2018), and then BERT4Rec utilized bidirectional context through a masked item prediction objective (Sun et al., 2019). These papers have

consolidated SR as a major research direction in recommender systems.

In practice, SR remains challenging because interaction sequences can be very sparse, short, long, or heterogeneous, which complicates accurate modeling. Short sequences contain little evidence about user preferences, while long sequences may include irrelevant interactions that dilute the user’s current intent. Moreover, variable time gaps between interactions and differing action semantics increase the difficulty of balancing short-term and long-term interests. Consequently, recent work emphasizes more powerful sequence encoders, hybrid architectures that combine global and local signals, incorporation of side information, and integration with large language models to improve robustness and representation quality (Pan et al., 2025).

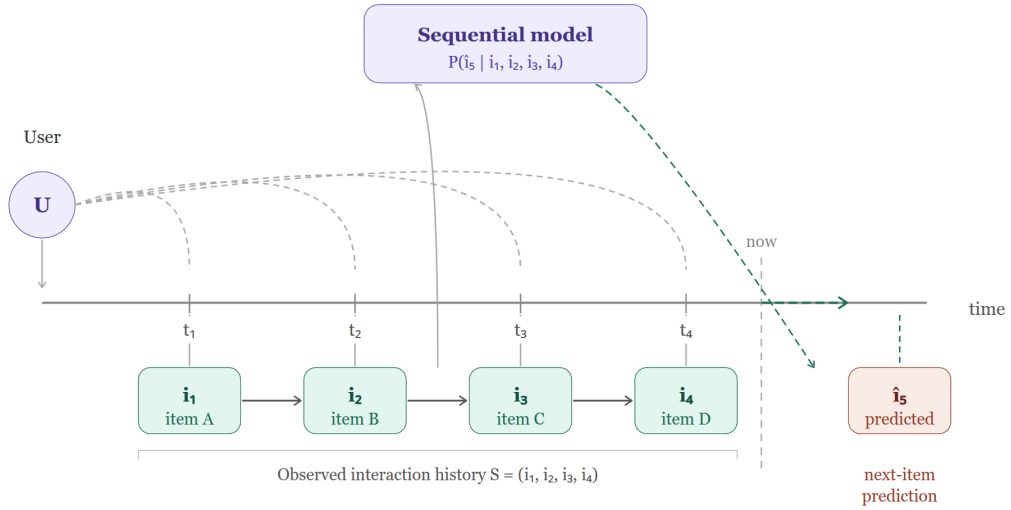


Figure 1.2: Illustration of a user interaction timeline for next-item prediction.

1.3 Sequence-Based Models

Sequence-based models constitute an important class of sequential recommendation methods that explicitly exploit the ordered nature of user interaction histories. Unlike simpler approaches that treat a user’s past actions as an unordered set, these models learn patterns from the exact sequence of events in order to predict the next likely item. This family of methods formed one of the earliest and most influential directions in sequential recommendation, and it includes recurrent, convolutional, and Transformer-based architectures.

1.3.1 GRU4Rec

GRU4Rec was one of the first deep learning models proposed for session-based recommendation and showed that recurrent neural networks can effectively model sequential user behavior (Hidasi et al., 2016). The model uses gated recurrent units to process interaction sequences in order, allowing it to capture short-term dependencies and session-level dynamics. Its key idea is that the hidden state summarizes the recent behavior of the user, which is then used to predict the next item in the sequence. This made GRU4Rec an important baseline for later sequential recommendation research.

1.3.2 Caser

Caser introduces a convolutional approach to sequential recommendation by embedding a user’s recent interaction history into a two-dimensional representation and applying convolutional filters over it (Tang and Wang, 2018). The model is designed to capture both general user preferences and local sequential patterns through vertical and horizontal convolutions. In contrast to recurrent models, Caser views the sequence as a structured pattern detection problem, which allows it to learn influential subsequences and short-range dependencies. This makes Caser a representative CNN-based baseline in the SR literature.

1.3.3 SASRec

SASRec applies self-attention to sequential recommendation, using a Transformer-style architecture to model long-range dependencies between interacted items (Kang and McAuley, 2018). Instead of processing the sequence strictly step by step, the model learns which past items are most relevant to the current prediction through attention weights. This gives SASRec a strong ability to capture both short- and long-term behavioral patterns while remaining efficient to train. It became one of the most influential Transformer-based baselines in sequential recommendation.

1.3.4 BERT4Rec

BERT4Rec extends the Transformer idea by adopting a bidirectional encoding strategy and a masked item prediction objective (Sun et al., 2019). Rather than predicting the next item in a strictly left-to-right fashion, the model randomly masks items in the sequence and learns to reconstruct them from both left and right context. This training objective encourages richer sequence representations and often improves the quality of learned user behavior embeddings. BERT4Rec is widely used as a strong benchmark for sequential recommendation due to its ability to model contextual dependencies more deeply.

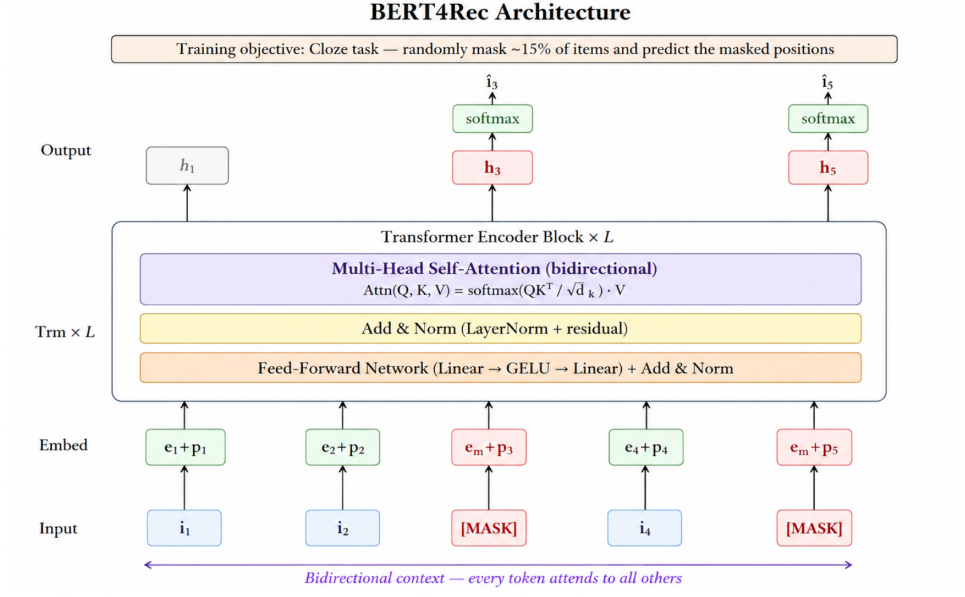


Figure 1.3: Architecture illustration of a sequence-based model such as SASRec or BERT4Rec.

Model	Approach	Key idea
GRU4Rec	Recurrent neural network	Models the interaction sequence step by step with gated recurrent units.
Caser	Convolutional neural network	Learns local sequential patterns from a sequence-as-image representation.
SASRec	Self-attention / Transformer encoder	Uses attention to focus on the most relevant past items for next-item prediction.
BERT4Rec	Bidirectional Transformer encoder	Learns contextual sequence representations using masked item prediction.

Table 1.1: Representative sequence-based recommendation models.

1.4 Graph-Based Models / GNN

Graph-based recommender systems model user-item interactions as a graph, where users and items are represented as nodes and historical interactions are encoded as edges. This formulation is useful because recommendation data are naturally relational: a user’s preference can be inferred not only from their own history, but also from their position in the interaction graph and from the neighborhoods of connected users and items. Graph neural networks therefore provide a powerful way to propagate collaborative signals, capture higher-order connectivity, and learn richer embeddings than methods based solely on local sequence patterns (S. Wu et al., 2020; He et al., 2020).

1.4.1 LightGCN

LightGCN is one of the most influential graph-based recommendation models because it simplifies graph convolution to its essential component: neighborhood aggregation (He et al., 2020). Instead of using feature transformation and nonlinear activation, LightGCN directly propagates user and item embeddings over the bipartite interaction graph and combines the embeddings from multiple propagation layers to obtain the final representation. This makes the model both efficient and effective, and it has become a standard baseline for graph-based collaborative filtering. Its main strength is the ability to capture collaborative signals from the full interaction graph while remaining simple enough to train and interpret.

Model	Approach	Key idea
LightGCN	Graph collaborative filtering	Learns user and item embeddings by propagating them over the bipartite interaction graph with only neighborhood aggregation.

Table 1.2: Representative graph-based recommendation models.

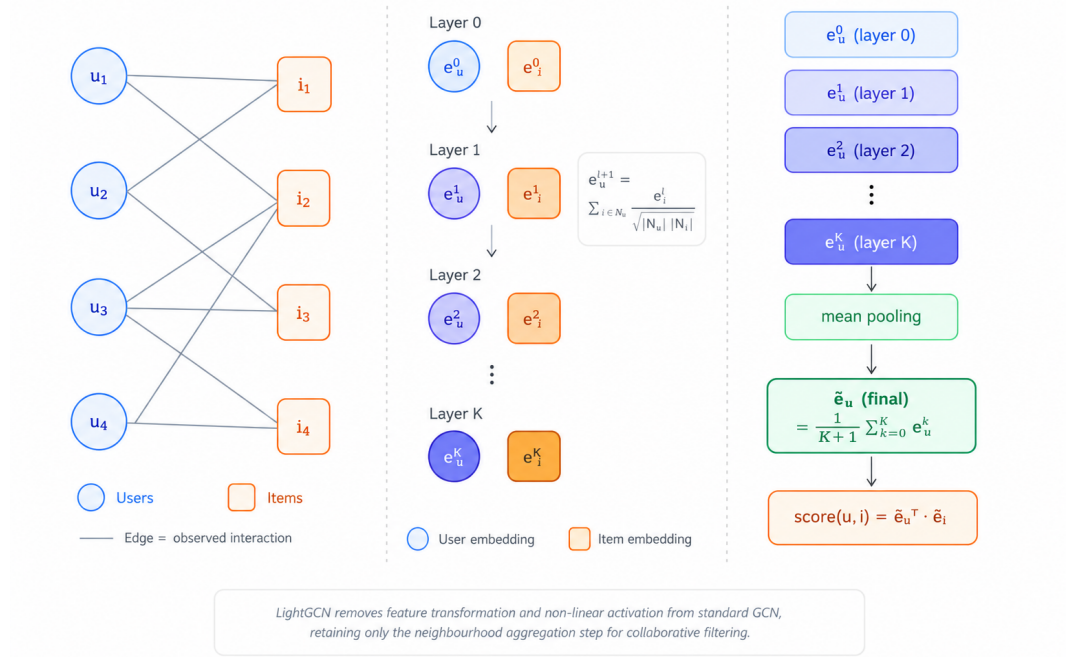


Figure 1.4: Illustration of a user-item bipartite graph and embedding propagation in LightGCN.

1.5 LLMs Applied to Sequential Recommendation

Large Language Models (LLMs) have recently entered recommendation systems by bringing strong semantic understanding of text, contextual reasoning, and transferable representations learned from large-scale corpora. In sequential recommendation, this is particularly valuable because many items are described by textual metadata such as titles, descriptions, reviews, or categorical attributes. Rather than relying only on item identifiers or interaction history, LLM-based methods can transform item text into dense embeddings that encode semantic similarity and can be injected into recommendation pipelines as initialization, side information, or direct ranking signals (Harte et al., 2023; Harte et al., 2024; L. Wu et al., 2024; Liu et al., 2025).

A central idea in this line of work is that the textual description of an item can act as a semantic bridge between sparse interaction data and richer domain knowledge. When item descriptions are encoded by an LLM, the resulting embeddings can capture not only surface-level wording, but also relations between concepts, product properties, and user-facing meaning. These embeddings can then be used to improve sequential recommenders by initializing item representations, enhancing content-based signals, or supporting hybrid models that combine textual and behavioral information. This is especially helpful in cold-start or sparse-data settings, where item IDs alone provide little information about the item’s semantics.

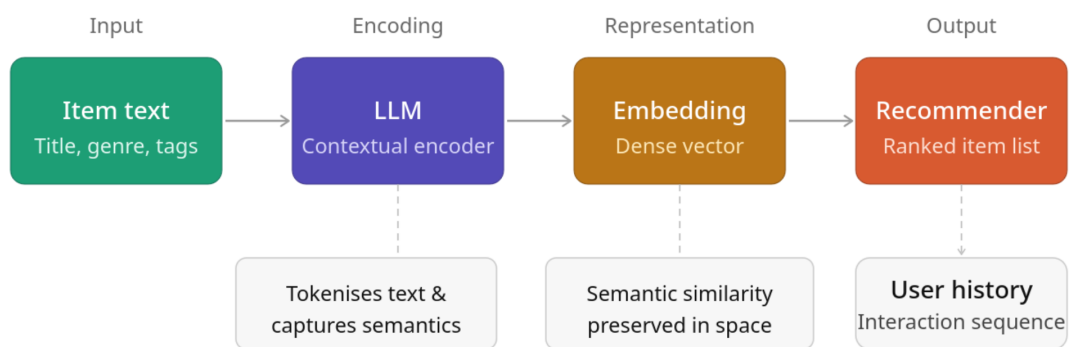


Figure 1.5: Pipeline for using item descriptions to generate embeddings with an LLM and feed them into a recommender.

Recent studies have shown that LLMs can be used in several ways for sequential recommendation. Some approaches initialize strong sequential models such as BERT4Rec

or SASRec with LLM-derived embeddings, while others fine-tune the LLM directly for recommendation or use it as a semantic encoder for items and user profiles (Harte et al., 2023; Harte et al., 2024; L. Wu et al., 2024). Survey work also highlights a broader shift from treating LLMs only as feature extractors toward using them as integrated components of recommendation pipelines, including knowledge enhancement, interaction enhancement, and model enhancement (Liu et al., 2025). This makes LLMs a promising direction for improving both representation quality and generalization in sequential recommendation.

At the same time, the usefulness of LLMs depends strongly on how item text is constructed and presented. A short product title, a structured attribute list, and a longer user-centric description can lead to different embedding spaces and therefore different recommendation behaviors. For this reason, item text design is not a trivial preprocessing step, but an important modeling choice that can influence semantic richness, distinctiveness, and downstream performance. This observation motivates the empirical study carried out in this memoir, where different item description strategies are analyzed in the context of sequential recommendation.

Use case	Method	Key idea
Embedding initialization	LLM $\rightarrow$ item vectors	Use semantic embeddings to initialize or enrich sequential recommenders.
Direct recommendation	LLM as recommender	Fine-tune or prompt the LLM to rank candidate items directly.
Hybrid enhancement	LLM + sequential model	Combine textual embeddings with behavioral sequence modeling.

Table 1.3: Examples of how LLMs are used in sequential recommendation.

1.6 Evaluation Metrics

To evaluate the performance of sequential recommendation models, this memoir uses three standard ranking-based metrics: Hit Ratio at K (HR@K), Normalized Discounted Cumulative Gain at K (NDCG@K), and Mean Reciprocal Rank (MRR). These metrics measure how well the model places the true next item among its top-ranked predictions. Since sequential recommendation is typically a top- K ranking problem, such metrics are more informative than simple classification accuracy.

1.6.1 Hit Ratio at K

HR@K measures whether the ground-truth next item appears in the top- K recommended items. It gives a binary reward of 1 if the target item is present in the rec-

ommendation list and 0 otherwise. As a result, HR@K captures the model’s ability to retrieve the correct item within a limited number of suggestions, but it does not consider the exact rank position inside the top- K .

1.6.2 Normalized Discounted Cumulative Gain at K

NDCG@K evaluates both whether the correct item is retrieved and how high it is ranked in the recommendation list. Higher-ranked correct predictions receive larger scores, which makes NDCG@K more sensitive to the ordering quality of the top- K list. This metric is especially useful when the position of the predicted item matters, which is often the case in practical recommendation scenarios.

1.6.3 Mean Reciprocal Rank

MRR measures the inverse of the rank position of the first relevant item. Unlike HR@K, it rewards models that place the target item very close to the top of the list. If the correct item is ranked first, the reciprocal rank is 1; if it is ranked second, it is $1/2$, and so on. MRR therefore reflects how early the desired item appears in the ranked output.

Metric	Formula	What it measures
HR@K	$\text{HR@K} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\text{rank}_i \leq K)$	Whether the true next item appears in the top- K recommendations.
NDCG@K	$\text{NDCG@K} = \frac{1}{N} \sum_{i=1}^N \frac{\mathbb{I}(\text{rank}_i \leq K)}{\log_2(\text{rank}_i + 1)}$	Retrieval quality with a stronger reward for higher-ranked correct items.
MRR	$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}$	How early the first relevant item appears in the ranked list.

Table 1.4: Ranking Metrics for Sequential Recommendation.

1.7 Conclusion

This chapter introduced the fundamental concepts underlying recommendation systems and provided an overview of the main paradigms used in practice, including collaborative filtering, sequence-based models, graph-based models, and LLM-enhanced approaches. It also presented the sequential recommendation setting, the principal baseline architectures used in this work, and the evaluation metrics employed to assess model performance.

Despite the progress made by these methods, several limitations remain, particularly in the way they model heterogeneous user histories, combine global and local signals, and exploit textual item information. These open questions motivate the research gaps addressed in the following chapter.

1.8 Identified Gaps and Positioning of Contributions

Despite the substantial progress achieved by recent sequential recommendation methods, two important gaps remain insufficiently addressed in the literature. First, many hybrid GNN-Transformer approaches combine global graph signals and local sequential signals using a fixed or uniform fusion strategy, without considering that users exhibit different interaction history profiles. In practice, users with sparse histories benefit more from global collaborative information, whereas users with rich histories can better exploit detailed sequential patterns. This observation motivates our first contribution, which introduces a length-adaptive fusion mechanism that dynamically balances graph-based and sequence-based representations according to the amount of available user history.

Second, although large language models have recently been integrated into recommendation systems, the influence of item textual description strategy on the quality of the resulting embeddings has not yet been systematically explored. Different forms of item text, such as titles, structured attributes, or user-oriented descriptions, may produce embeddings with different semantic properties and downstream effects on recommendation performance. This gap motivates our second contribution, which conducts a systematic study of how item description design affects LLM embeddings in sequential recommendation.

Together, these two gaps define the positioning of this memoir: the first contribution addresses the adaptive fusion of structural and sequential signals, while the second investigates the semantic role of item text in LLM-based sequential recommendation.

Length-Adaptive Hybrid GNN-Transformer for Sequential Recommendation

2.1 Introduction

Sequential recommendation remains a challenging problem because user interaction histories are heterogeneous in both length and density. Transformer-based models such as SASRec (Kang and McAuley, 2018) and BERT4Rec (Sun et al., 2019) have demonstrated strong performance by capturing long-range dependencies within interaction sequences, but they rely on the availability of sufficiently long histories to generalize reliably. Graph Neural Network models such as LightGCN (He et al., 2020) address this by aggregating global collaborative signals from the full user-item interaction graph, making them more robust for sparse users. However, when used in isolation, each paradigm captures only one side of the recommendation signal: sequence models ignore global item co-occurrence structure, while graph models ignore the fine-grained temporal order of individual user interactions (S. Wu et al., 2020; Pan et al., 2025).

This chapter proposes a Length-Adaptive Hybrid GNN-Transformer, a model that combines both paradigms through a user-specific fusion mechanism controlled by the length of each user’s interaction history. The core idea is that the relative reliability of the graph signal versus the sequence signal is not constant across users: it varies directly with the amount of available sequential evidence. The proposed model quantifies this variation through a scalar weight $\lambda(u)$ that dynamically adjusts the contribution of the GNN and Transformer branches for each user individually. The remainder of this chapter is organized as follows: Section 2.2 discusses the limitations of existing models and formally states the research problem; Section 2.3 introduces the adaptive fusion mechanism and the formal problem definition; Section 2.4 presents the family of hybrid model variants; and Section 2.5 concludes the chapter.

2.2 Limitations of Existing Models and Problem Statement

Despite the progress achieved by the models reviewed in Chapter 1, three concrete limitations can be identified in the existing literature on hybrid GNN-Transformer sequential recommendation.

Limitation 1 - Fixed fusion weights in existing hybrid models. Hybrid approaches that combine GNN and Transformer components, such as those surveyed in (S. Wu et al., 2020), typically apply a single global fusion weight that is shared uniformly across all users. This fixed weighting strategy treats every user identically, regardless of how much interaction history is available. As a result, the model cannot adapt to the fact that the relative informativeness of the graph signal versus the sequence signal varies substantially from one user to another. A fixed weight that is optimal on average will systematically underserve both sparse users and dense users (Pan et al., 2025).

Limitation 2 - Transformer-only models fail sparse users. Self-attention mechanisms, as used in SASRec (Kang and McAuley, 2018) and BERT4Rec (Sun et al., 2019), require a sufficient number of interactions to produce reliable contextualized representations. When a user has only a small number of past interactions, the self-attention layers lack enough context to generalize, and the resulting sequence embedding is noisy or underspecified. These models therefore provide poor recommendations for sparse and cold-start users, a critical segment in any real-world recommendation platform.

Limitation 3 - GNN-only models waste dense users’ sequential patterns. Graph-based models such as LightGCN (He et al., 2020) aggregate neighborhood information over the full interaction graph and produce strong collaborative embeddings for sparse users. However, they do not model the temporal order of a user’s individual interactions. For users with long, rich interaction histories, this means that fine-grained sequential patterns - the exact order in which items were consumed, recency effects, and evolving preferences - are entirely discarded. The graph signal alone is insufficient to capture dynamic intent for active users (Pan et al., 2025).

Problem Statement

The three limitations above converge on a single fundamental question, which constitutes the first research problematic of this thesis:

How to dynamically adapt the fusion between global graph signals and local sequential modelling according to each user’s interaction history profile?

Answering this question requires a model that is aware of each user’s interaction density and uses this information to balance the GNN and Transformer contributions at the individual user level, rather than applying a fixed global weight. This is the design principle that motivates the proposed architecture described in the following section.

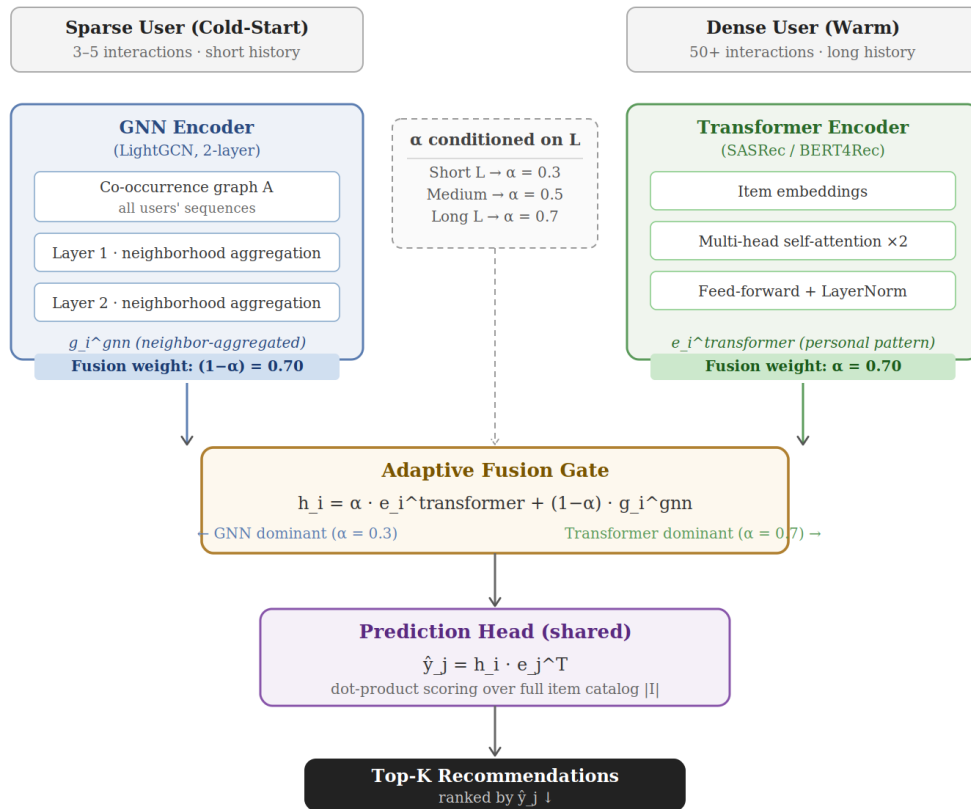


Figure 2.1: Comparison of sparse and dense user history profiles and their ideal information sources.

2.3 Proposed Solution: The Adaptive Fusion Mechanism

2.3.1 Formal Problem Definition

Let $\mathcal{U}$ and $\mathcal{I}$ denote the sets of users and items, respectively. For each user $u \in \mathcal{U}$, let $S_u = (i_1, i_2, \dots, i_{n_u})$ be their interaction history of length n_u , where each $i_t \in \mathcal{I}$ is an item consumed at time step t , ordered chronologically. The task of next-item sequential recommendation is defined as: given the prefix $S_u^{(t)} = (i_1, \dots, i_t)$, learn a scoring function

$$f_{\Theta}(u, S_u^{(t)}, j) \in \mathbb{R} \quad (2.1)$$

that assigns a relevance score to each candidate item $j \in \mathcal{I}$, where Θ denotes all trainable parameters. At inference time, items are ranked by their scores and top- K accuracy is evaluated via HR@ K , NDCG@ K , and MRR@ K as defined in Chapter 1.

2.3.2 The Adaptive Fusion Mechanism

The proposed model computes two independent representations for each user u : a graph-based representation $\mathbf{h}_u^{\text{GNN}}$, produced by a Graph Neural Network encoder operating over the global item co-occurrence graph, and a sequence-based representation $\mathbf{h}_u^{\text{Trans}}$, produced by a Transformer encoder operating over the user's personal interaction history S_u . These two representations are then combined through a user-specific scalar weight $\lambda(u) \in [0, 1]$:

$$\mathbf{h}_u = \lambda(u) \cdot \mathbf{h}_u^{\text{Trans}} + (1 - \lambda(u)) \cdot \mathbf{h}_u^{\text{GNN}} \quad (2.2)$$

where the fusion weight $\lambda(u)$ is derived from the user's interaction history length n_u and a normalization factor $n_{\max}$, the maximum history length observed in the training set:

$$\lambda(u) = \sigma\left(\frac{n_u}{n_{\max}}\right) \quad (2.3)$$

and $\sigma(\cdot)$ denotes the sigmoid function $\sigma(x) = 1/(1 + e^{-x})$.

2.3.3 Intuition and Rationale

The intuition behind Equations 2.2 and 2.3 is straightforward. When a user has a short interaction history (n_u is small), $\lambda(u)$ is close to 0, and the fused representation $\mathbf{h}_u$ relies predominantly on $\mathbf{h}_u^{\text{GNN}}$. This is desirable because the GNN aggregates global co-occurrence signals from all users in the training set and is therefore informative even when individual sequence evidence is scarce. Conversely, when a user has a long interaction history (n_u is large), $\lambda(u)$ approaches 1, and $\mathbf{h}_u$ relies predominantly on $\mathbf{h}_u^{\text{Trans}}$. In this regime, the Transformer can exploit the full richness of the user’s personal sequential patterns, which carry more information than global averages.

The use of history length n_u as the proxy variable is motivated by the observation that the reliability of self-attention degrades when the input sequence is too short (Kang and McAuley, 2018; Sun et al., 2019), while GNN neighborhood aggregation is length-independent by construction (He et al., 2020). The sigmoid normalization in Equation 2.3 ensures that $\lambda(u)$ remains bounded in $[0, 1]$ and transitions smoothly between the two extremes, avoiding hard discontinuities in the embedding space.

2.4 Family of Proposed Hybrid Models

This work proposes four hybrid models, all built on the same core idea: combining a graph encoder with a sequential encoder, and blending their outputs based on how long a user’s interaction history is. The models share the same LightGCN (He et al., 2020) graph backbone and differ in how they encode sequences and how they control the blending.

2.4.1 Hybrid BERT4Rec + LightGCN with Fixed Fusion

The simplest variant applies a single fixed blending weight to all users equally. LightGCN provides collaborative signals from the item co-occurrence graph, while BERT4Rec (Sun et al., 2019) encodes the interaction sequence using bidirectional self-attention. The contribution of each encoder is set manually before training and never changes. This model serves as the baseline of the family: it confirms that combining the two encoders helps, but it does not adapt to differences between users.

2.4.2 Hybrid BERT4Rec + LightGCN with Discrete Fusion

The discrete variant introduces length-awareness by dividing users into three groups based on their history length: short, medium, and long. Each group receives a different fixed blending weight, giving short-history users more graph signal and

long-history users more sequential signal. The thresholds and weights are set manually and stay fixed during training. Compared to the fixed model, this variant is more principled because it acknowledges that different users need different encoder balances, while remaining fully interpretable and adding no trainable parameters.

2.4.3 Hybrid BERT4Rec + Temporal Graph Transformer (TGT)

This variant replaces LightGCN with a Temporal Graph Transformer, which extends standard graph attention with time-awareness. Concretely, it augments the attention keys with learned time encodings so that the model can weigh recent interactions more heavily than older ones when building item representations. BERT4Rec handles the sequential side as before. The two branches are fused through a single learnable gating parameter that is optimised during training, allowing the model to find the right balance automatically. This makes TGT-BERT4Rec the most expressive graph-based hybrid in the family, at the cost of higher complexity.

2.4.4 Hybrid BERT4Rec + Temporal Convolutional Network (TCN)

The TCN variant replaces the graph encoder entirely with a Temporal Convolutional Network. TCN uses stacked dilated causal convolutions, meaning it can capture long-range sequential dependencies without ever looking at future tokens. Because convolutions process sequences much faster than attention, this model is significantly more efficient than the transformer-only alternatives. The TCN and BERT4Rec branches share the same item embeddings and their outputs are combined through either a fixed weight, a learnable scalar, or a learned projection layer that concatenates both representations. This variant is particularly useful when inference speed or memory is a constraint.

Table 2.1: Overview of the four proposed hybrid model variants.

Model	Graph / Conv encoder	Seq. encoder	Fusion
Fixed Fusion	LightGCN	BERT4Rec	Fixed scalar, same for all users
Discrete Fusion	LightGCN	BERT4Rec	Fixed scalar per length group
TGT-BERT4Rec	Temporal Graph Transformer	BERT4Rec	Learned gate
TCN-BERT4Rec	Temporal Conv. Network	BERT4Rec	Fixed / learned / concat

2.5 Conclusion

This chapter proposed a Length-Adaptive Hybrid GNN-Transformer framework for sequential recommendation, built around the core idea that the optimal balance between global graph signals and local sequential signals is not constant but varies with each user’s interaction history length. The novelty of the proposed approach lies in the user-specific fusion weight $\lambda(u)$, which dynamically shifts the representation from GNN-dominated for sparse users toward Transformer-dominated for dense users, addressing a design flaw shared by all existing fixed-weight hybrid models (S. Wu et al., 2020; Pan et al., 2025). Two concrete variants were derived from this framework by combining LightGCN (He et al., 2020) with BERT4Rec (Sun et al., 2019), and using either a linear or sigmoid fusion schedule, allowing the effect of the fusion schedule choice to be studied independently. A current limitation of the framework is that $\lambda(u)$ is computed from history length alone using a fixed formula, rather than being learned end-to-end from data, which may leave performance gains on the table for users whose optimal fusion point does not align perfectly with their history length. The experimental validation of both variants, including comparisons against the baseline models from Chapter 1 and a per-user-segment analysis, is presented in Chapter 3.

Experiments and Results: Length-Adaptive Hybrid GNN-Transformer

3.1 Introduction

Chapter 2 proposed a Length-Adaptive Hybrid GNN-Transformer framework for sequential recommendation, built around the idea that the relative contribution of global graph signals and local sequential signals should vary according to each user’s interaction history length. The framework was instantiated into two concrete model variants combining LightGCN (He et al., 2020) with BERT4Rec (Sun et al., 2019), using either a linear or sigmoid fusion schedule for $\lambda(u)$. This chapter provides the empirical validation of these two variants against a set of strong sequential and graph-based baselines on two standard benchmark datasets.

The experiments in this chapter are designed to answer three concrete questions. First, does the proposed adaptive fusion mechanism yield better overall recommendation performance compared to fixed-weight hybrid baselines and to single-paradigm models? Second, does the length-adaptive design specifically improve performance for sparse users, who are systematically underserved by Transformer-only models? Third, does each component of the proposed architecture - the GNN branch, the Transformer branch, and the adaptive $\lambda(u)$ schedule - contribute independently to the final performance, or can any of them be removed without degrading results?

The remainder of this chapter is organized as follows. Section 3.2 describes the datasets used and justifies their selection. Section 3.3 details the preprocessing pipeline. Section 3.4 reports all hyperparameter settings. Section 3.5 presents the baseline models and the evaluation protocol. Section 3.6 reports and analyzes the main results. Section 3.7 presents the ablation study. Finally, Section 3.8 concludes the chapter and transitions to Chapter 4.

3.2 Datasets

The experiments are conducted on two standard sequential recommendation benchmarks derived from the MovieLens collection (Harper and Konstan, 2015), which are among the most widely used datasets in the sequential recommendation literature (Pan et al., 2025; Kang and McAuley, 2018; Sun et al., 2019). These

datasets were chosen because they cover two different scales of interaction density, allowing the evaluation of model robustness across both sparse and dense user populations - a property that is particularly important for validating the length-adaptive design of the proposed model.

MovieLens-100K (ML-100K) contains 100,000 explicit ratings collected from 943 users on 1,682 movies. After filtering users with fewer than 5 interactions and converting ratings to implicit feedback (an interaction is recorded for any rating ≥ 1), the dataset yields 938 test users. This smaller dataset contains a higher proportion of sparse users relative to ML-1M, making it a valuable testbed for evaluating cold-start and sparse-user behavior.

MovieLens-1M (ML-1M) contains approximately 1 million ratings from 6,040 users on 3,706 movies. After the same preprocessing steps, the dataset yields 6,034 test users. Its larger scale and denser interaction distribution make it a complementary benchmark to ML-100K, allowing the evaluation of model scalability and performance under richer sequential evidence.

Table 3.1: Statistics of the datasets used in this chapter after preprocessing.

Dataset	#Users	#Items	#Interactions	Sparsity	Avg. seq. length
ML-100K	943	1,682	100,000	93.70%	106.0
ML-1M	6,040	3,706	1,000,209	95.53%	165.5

Sparsity is computed as $1 - \frac{\#Interactions}{\#Users \times \#Items}$. Both datasets confirm the typical sparsity challenge in collaborative filtering, motivating the use of GNN signals to supplement sequential evidence for users with short histories.

3.3 Preprocessing

The raw MovieLens datasets are preprocessed following the standard pipeline used in the sequential recommendation literature (Kang and McAuley, 2018; Sun et al., 2019; Pan et al., 2025). The preprocessing steps are applied identically to both ML-100K and ML-1M to ensure fair comparison across datasets. The full pipeline is illustrated in Figure 3.1.

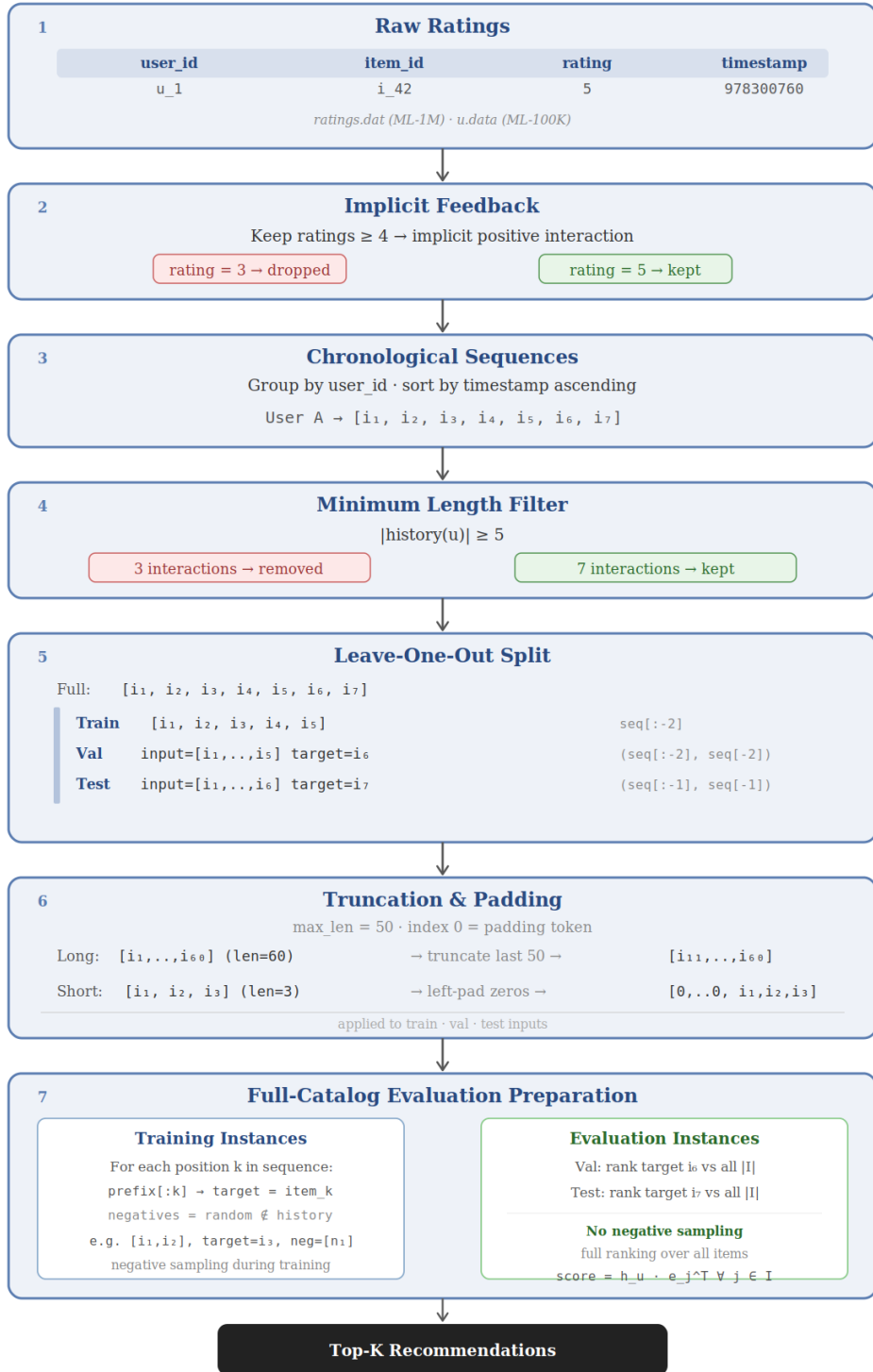


Figure 3.1: Preprocessing pipeline applied to both datasets.

Step 1 - Implicit feedback conversion. All explicit ratings are converted to

implicit feedback: an interaction is recorded if and only if the rating value is greater than or equal to 1. This binary conversion is standard in sequential recommendation, where the goal is to model the order of consumed items rather than preference intensity.

Step 2 - 5-core filtering. Users and items with fewer than 5 interactions are removed from the dataset. This filtering step ensures that every user has enough interaction history to construct a meaningful training sequence, and that every item appears sufficiently often to learn a reliable embedding. The filter is applied iteratively until no user or item falls below the threshold.

Step 3 - Chronological sorting. For each user, all interactions are sorted in ascending order of timestamp. This produces a chronologically ordered interaction sequence $S_u = (i_1, i_2, \dots, i_{n_u})$ for each user u , which is the input format required by all sequential models.

Step 4 - Leave-one-out split. The dataset is split into training, validation, and test sets following the leave-one-out protocol (Kang and McAuley, 2018). For each user, the last item in the interaction sequence is held out for testing, the second-to-last item is held out for validation, and all remaining items form the training set. This protocol is standard in the sequential recommendation literature and ensures that the temporal order of interactions is preserved.

Step 5 - Sequence truncation. To manage computational cost, interaction sequences are truncated to a maximum length of 200 for hybrid models and 50 for pure sequential baselines. When a sequence exceeds the maximum length, the most recent interactions are retained and earlier ones are discarded, preserving the most informative recent context.

Step 6 - Negative sampling for evaluation. During evaluation, each positive test item is ranked against all items in the full item catalog. No negative sampling is applied at test time, ensuring that the reported metrics reflect full-catalog ranking performance rather than sampled approximations. During training, 100 negative items are sampled uniformly at random per positive interaction for the binary cross-entropy loss.

3.4 Hyperparameters

All models are implemented and trained under identical conditions to ensure fair comparison. The hyperparameters used in this chapter are reported in Table 3.2. General architecture hyperparameters - embedding dimension, number of attention

heads, number of Transformer blocks, and dropout rate - are fixed to the same values across all models. Training hyperparameters - optimizer, learning rate, batch size, and early stopping criterion - are also held constant. This design ensures that any observed performance differences between models are attributable to architectural choices rather than training configuration differences.

The GNN-specific hyperparameters - number of GCN layers and co-occurrence graph construction parameters - are fixed based on standard settings from the LightGCN literature (He et al., 2020). The co-occurrence window size $w = 5$ and minimum co-occurrence threshold $\tau = 2$ were chosen to balance graph density and edge quality: a window that is too narrow misses meaningful item relationships, while a threshold that is too low introduces noisy edges that degrade embedding quality.

For the HybridDiscrete variant, the three fusion weight values α_{short} , α_{mid} , and α_{long} are treated as hyperparameters tuned on the validation set using NDCG@10 as the selection criterion. The history length thresholds $L_{\text{short}}^* = 10$ and $L_{\text{long}}^* = 30$ partition users into three bins and were selected based on the empirical distribution of history lengths in the training set.

3.5 Baselines

The proposed hybrid model variants are evaluated against five baseline models grouped into three categories: sequential-only models, a graph-only model, and hybrid ablation variants derived from the proposed framework itself. All models are evaluated using the leave-one-out protocol described in Section 3.3, where each positive test item is ranked against all items in the full catalog. Performance is reported at cut-off $K \in \{5, 10, 20\}$ for HR@ K , NDCG@ K , and MRR@ K .

Sequential-Only Baselines

GRU4Rec (Hidasi et al., 2016) is a recurrent neural network model that uses gated recurrent units to process interaction sequences step by step, capturing short-term session-level dynamics. It serves as the recurrent baseline and represents the earliest class of deep sequential recommendation models.

SASRec (Kang and McAuley, 2018) applies unidirectional causal self-attention to model long-range dependencies in interaction sequences. It is trained with binary cross-entropy loss and is one of the strongest and most widely adopted Transformer-based baselines in sequential recommendation.

BERT4Rec (Sun et al., 2019) uses bidirectional self-attention and a masked item

Table 3.2: Hyperparameter settings for all models.

Hyperparameter	Value	Search range / Note
<i>General</i>		
Embedding dimension d	64	Fixed
Attention heads	2	Fixed
Transformer blocks B	2	Fixed
Dropout rate	0.2	Fixed
Max sequence length (hybrid)	200	Fixed
Max sequence length (base-lines)	50	Fixed
<i>Training</i>		
Optimizer	Adam	Fixed
Learning rate η	0.001	Fixed
Batch size	256	Fixed
Max epochs	200	Fixed
Early stopping patience	20	Validation NDCG@10
Negatives per positive	100	Fixed
<i>GNN Encoder</i>		
GCN layers L_{GNN}	2	Fixed
Co-occurrence window w	5	Fixed
Min co-occurrence threshold τ	2	Fixed
<i>HybridDiscrete Fusion</i>		
Short-history threshold L_{short}^*	10	Fixed
Long-history threshold L_{long}^*	30	Fixed
α_{short}	0.3	Tuned on validation set
α_{mid}	0.5	Tuned on validation set
α_{long}	0.7	Tuned on validation set

prediction objective to produce rich contextual sequence representations. It is the sequential backbone shared by all proposed hybrid variants and serves as the primary baseline against which the hybrid framework is evaluated.

Caser (Tang and Wang, 2018) applies horizontal and vertical convolutional filters over a sequence-as-image representation to capture both point-level and union-level sequential patterns. It represents the convolutional family of sequential models.

Graph-Only Baseline

LightGCN (He et al., 2020) simplifies graph convolution to pure neighborhood aggregation over the user-item bipartite graph, without feature transformation or nonlinear activation. It is included not as a competitive sequential baseline but as a reference point that isolates the contribution of pure collaborative filtering signals in a next-item prediction setting. Its relatively low performance on sequential metrics

directly motivates the hybrid design: static graph-based collaborative filtering alone is insufficient when temporal ordering matters.

Hybrid Ablation Variants

The following three variants of the proposed framework are included as internal ablation baselines to isolate the contribution of each architectural component:

HybridFixed uses a single global fusion weight α shared uniformly across all users, trained end-to-end as a learnable scalar. It verifies that combining GNN and Transformer embeddings is beneficial, while treating every user identically regardless of history length.

DCH-BERT4Rec replaces the GCN graph encoder with a dilated causal convolutional network that encodes local sequential transition patterns. It tests whether local convolutional features provide complementary information to the global self-attention of BERT4Rec, without relying on a pre-built item graph.

TGT-BERT4Rec augments the item co-occurrence graph with temporal information by encoding interaction timestamps as continuous time embeddings, and combines the temporal graph encoder output with BERT4Rec using a learnable gated fusion. It is particularly suited to short-history users whose temporal patterns encode strong preference signals even when interaction counts are low.

Table 3.3: Summary of baseline models used in this chapter.

Model	Category	Key characteristic
GRU4Rec	Sequential	Gated recurrent units for session-level dynamics
Caser	Sequential	Convolutional filters over sequence-as-image
SASRec	Sequential	Unidirectional causal self-attention
BERT4Rec	Sequential	Bidirectional masked item prediction
LightGCN	Graph	Pure neighborhood aggregation on bipartite graph
HybridFixed	Hybrid ablation	Fixed global fusion weight for all users
DCH-BERT4Rec	Hybrid ablation	Dilated convolution + BERT4Rec
TGT-BERT4Rec	Hybrid ablation	Temporal graph encoder + BERT4Rec

3.6 Results

Table 3.4 and Table 3.5 report the main results on ML-1M and ML-100K respectively. Bold values indicate the best result per column and underlined values indicate the second best. The proposed hybrid variants are marked with †. Results are reported at cut-off $K \in \{5, 10, 20\}$ for HR@K, NDCG@K, and MRR@K.

Table 3.4: Test performance on MovieLens-1M. Best results bold, second-best underlined. † = proposed hybrid.

Model	HR@K			NDCG@K			MRR@K		
	@5	@10	@20	@5	@10	@20	@5	@10	@20
GRU4Rec	0.0365	0.0708	0.1173	0.0214	0.0324	0.0440	0.0165	0.0210	0.0241
LightGCN	0.0121	0.0247	0.0467	0.0065	0.0106	0.0161	0.0047	0.0064	0.0079
Caser	0.0684	0.1260	0.2135	0.0397	0.0581	0.0801	0.0303	0.0379	0.0438
SASRec	0.0515	0.0979	0.1767	0.0298	0.0446	0.0643	0.0228	0.0287	0.0340
BERT4Rec	0.0757	0.1374	0.2274	0.0453	0.0652	0.0879	0.0354	0.0437	0.0498
DCH-BERT4Rec†	0.0754	0.1385	0.2325	0.0449	0.0652	0.0888	0.0350	0.0433	0.0498
TGT-BERT4Rec†	0.0820	<u>0.1480</u>	<u>0.2444</u>	0.0477	<u>0.0689</u>	<u>0.0930</u>	0.0365	<u>0.0452</u>	<u>0.0517</u>
HybridDiscrete†	0.0832	0.1427	0.2317	0.0492	0.0682	0.0905	0.0381	0.0459	0.0519
HybridFixed†	0.0852	0.1485	0.2511	0.0524	0.0727	0.0984	0.0417	0.0499	0.0569

Table 3.5: Test performance on MovieLens-100K. Best results bold, second-best underlined. † = proposed hybrid.

Model	HR@K			NDCG@K			MRR@K		
	@5	@10	@20	@5	@10	@20	@5	@10	@20
GRU4Rec	0.0341	0.0650	0.1077	0.0201	0.0302	0.0411	0.0156	0.0198	0.0229
LightGCN	0.0320	0.0682	0.1194	0.0196	0.0312	0.0441	0.0155	0.0202	0.0237
Caser	0.0576	0.1109	0.1844	0.0369	0.0540	0.0726	0.0302	0.0371	0.0422
SASRec	0.0416	0.0682	0.1461	0.0255	0.0339	0.0532	0.0202	0.0235	0.0286
BERT4Rec	0.0565	0.0949	0.1844	0.0327	0.0449	0.0671	0.0250	0.0299	0.0358
DCH-BERT4Rec†	0.0682	0.1194	0.2111	0.0386	<u>0.0551</u>	0.0780	0.0290	<u>0.0358</u>	0.0419
TGT-BERT4Rec†	0.0512	0.0981	0.1684	0.0340	0.0490	0.0665	0.0284	0.0345	0.0391
HybridDiscrete†	0.0512	0.0853	0.1535	0.0291	0.0402	0.0575	0.0219	0.0266	0.0313
HybridFixed†	0.0501	<u>0.0864</u>	<u>0.1727</u>	<u>0.0279</u>	0.0393	<u>0.0610</u>	<u>0.0207</u>	0.0252	<u>0.0311</u>

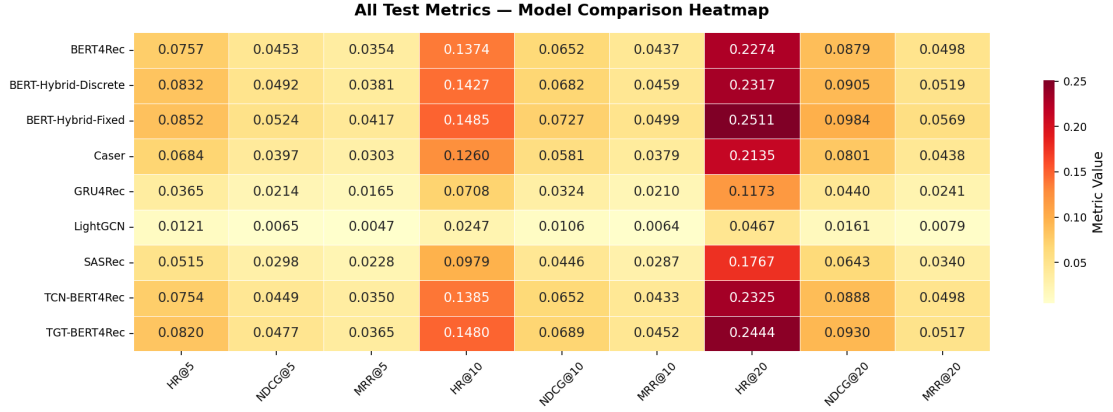


Figure 3.2: Test metrics heatmap on ML-1M. Darker cells indicate higher values. Hybrids marked with †.

Figure 3.2 confirms that all hybrid variants outperform their single-paradigm counterparts on ML-1M across every metric and cut-off. **HybridFixed** achieves the best overall results with $\text{HR@10} = 0.1485$, $\text{NDCG@10} = 0.0727$, and $\text{MRR@10} = 0.0499$, representing gains of +8.1%, +11.5%, and +14.2% respectively over BERT4Rec. **TGT-BERT4Rec** ranks second, recording the highest $\text{HR@20} = 0.2444$ in the entire comparison, suggesting that temporal graph signals are especially beneficial at wider cut-offs. **LightGCN** records the weakest performance overall ($\text{HR@10} = 0.0247$), directly motivating the hybrid design: pure graph-based collaborative filtering is insufficient when temporal ordering matters. The consistent darkening of cells across hybrid rows confirms that the performance advantage is stable across all metrics and cut-off values.

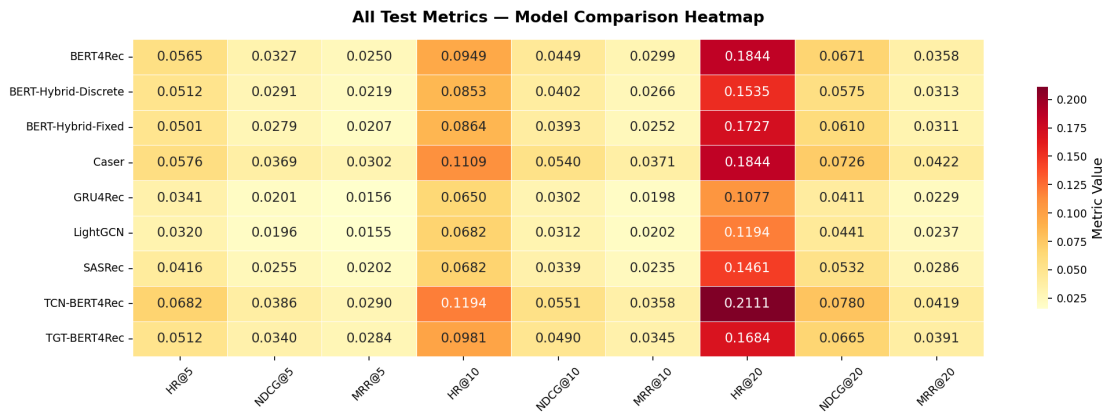


Figure 3.3: Test metrics heatmap on ML-100K. Darker cells indicate higher values. Hybrids marked with †.

On ML-100K, the ranking of models shifts compared to ML-1M. TCN-BERT4Rec achieves the best $\text{HR@10} = 0.1194$ and $\text{HR@20} = 0.2111$, suggesting that local

convolutional transition features are particularly effective on this smaller and denser dataset. BERT4Rec and Caser tie for the highest $\text{HR@20} = 0.1844$, while the hybrid variants HybridDiscrete and HybridFixed record lower overall scores than on ML-1M, indicating that the adaptive fusion mechanism is less decisive when interaction sequences are shorter and denser. LightGCN again records the weakest performance ($\text{HR@10} = 0.0682$), confirming that graph-only models struggle consistently in sequential settings regardless of dataset scale. The heatmap also reveals that all models achieve lower absolute values on ML-100K than on ML-1M, which is expected given the smaller number of users and the higher proportion of sparse interaction histories in this dataset.

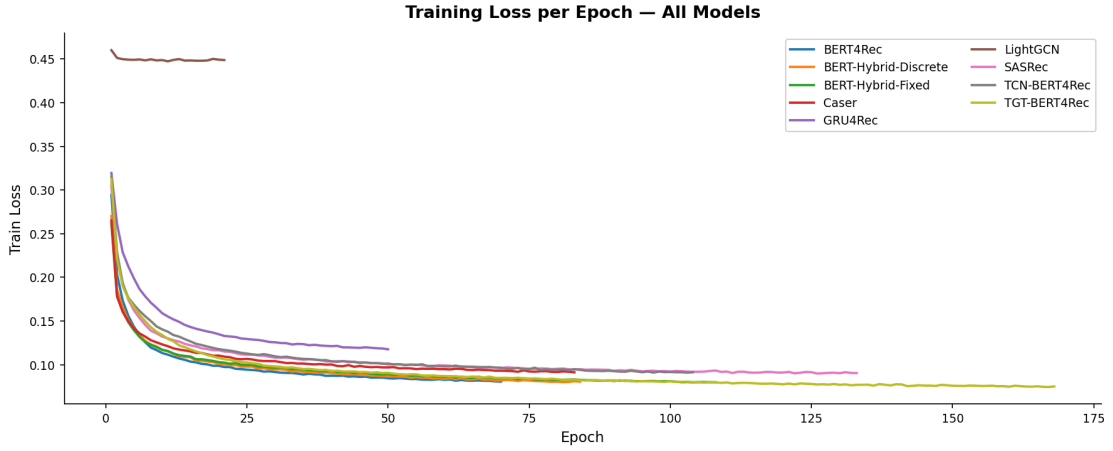


Figure 3.4: Training loss per epoch on ML-1M for all models.

Figure 3.4 reveals three distinct convergence behaviors on ML-1M. LightGCN converges immediately to a high loss floor of approximately 0.45 and remains flat throughout training, reflecting the fundamental limitation of its purely graph-based objective which does not model sequential order. GRU4Rec converges more slowly than all other models, reaching a stable loss only around epoch 50, which is consistent with the well-known difficulty of training recurrent networks on long sequences. All Transformer-based and hybrid models converge rapidly within the first 20 epochs and continue to decrease steadily, with TGT-BERT4Rec achieving the lowest final training loss near epoch 170. The close grouping of the hybrid variants with BERT4Rec and SASRec confirms that adding the GNN branch does not destabilize or slow down the training process.

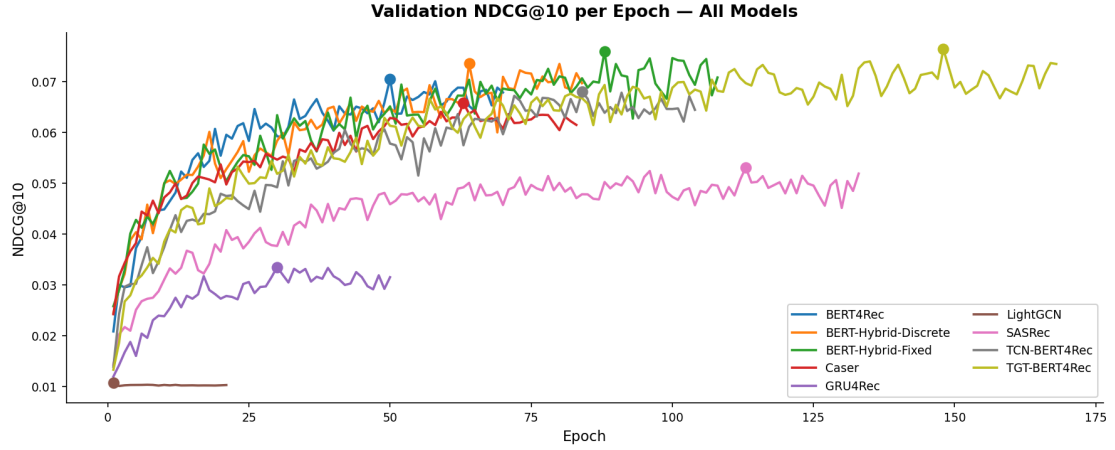


Figure 3.5: Validation NDCG@10 per epoch on ML-1M. Filled circles mark best checkpoint per model.

Figure 3.5 shows the validation NDCG@10 curves on ML-1M across all epochs, with filled circles marking the best checkpoint selected by early stopping. TGT-BERT4Rec reaches the highest peak validation NDCG@10 of approximately 0.075 at epoch 148, followed closely by BERT-Hybrid-Fixed which peaks near epoch 88. Most Transformer-based and hybrid models converge to a similar performance band between 0.062 and 0.072, indicating that the top-performing architectures are competitive throughout training rather than separated by a single lucky checkpoint. SASRec plateaus at a noticeably lower level around 0.050, suggesting that unidirectional attention is less effective than bidirectional or graph-augmented approaches on this dataset. GRU4Rec peaks early at epoch 30 with a validation NDCG@10 of only 0.033, confirming its limited capacity to model long interaction sequences. LightGCN remains flat near 0.010 throughout all epochs, further validating its unsuitability as a standalone sequential recommender.

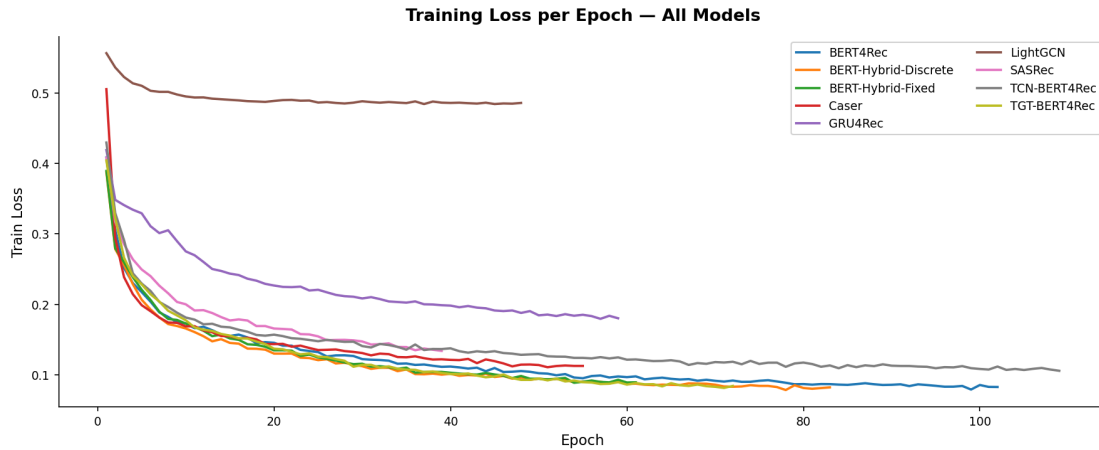


Figure 3.6: Training loss per epoch on ML-100K for all models.

Figure 3.6 shows that the convergence behavior on ML-100K closely mirrors that observed on ML-1M. LightGCN again remains at a high loss plateau of approximately 0.49 throughout all 100 epochs, consistent with its inability to model sequential patterns. GRU4Rec converges more slowly than all other models, stabilizing only around epoch 60 at a loss of 0.19, which is notably higher than the Transformer-based models. All remaining models converge rapidly within the first 10 epochs and continue to decrease steadily toward a final loss between 0.08 and 0.11. Notably, BERT-Hybrid-Discrete and TGT-BERT4Rec achieve the lowest final training loss values on this dataset, while TCN-BERT4Rec stabilizes slightly higher at around 0.10. The consistent convergence pattern across both datasets confirms that the hybrid architecture introduces no training instability and behaves reliably regardless of dataset scale.

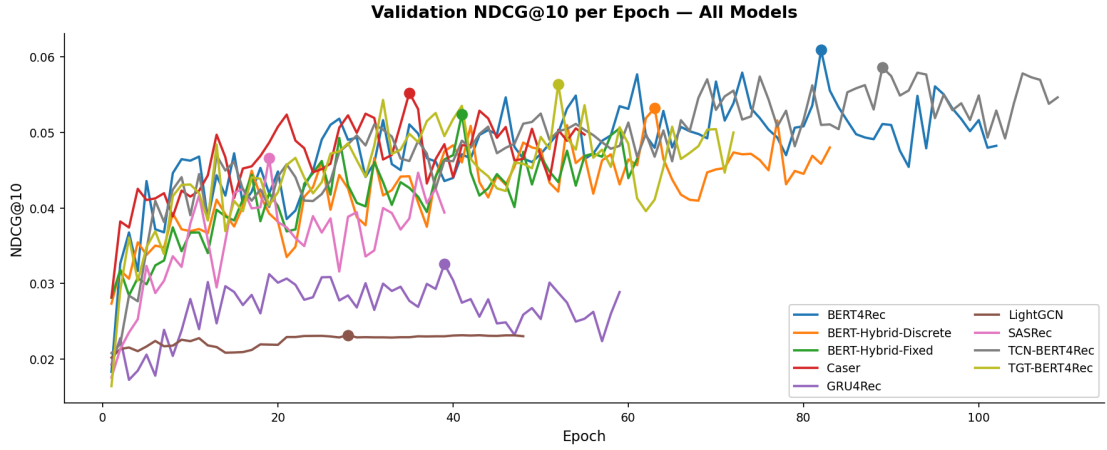


Figure 3.7: Validation NDCG@10 per epoch on ML-100K. Filled circles mark best checkpoint per model.

On ML-100K, BERT4Rec achieves the highest peak validation NDCG@10 of approximately 0.061, followed closely by TCN-BERT4Rec which maintains a stable plateau in the later epochs. Caser peaks early but then declines, suggesting overfitting on this smaller dataset. The hybrid variants show more volatile curves compared to ML-1M, which is expected given the reduced dataset size. GRU4Rec and LightGCN again rank last, confirming their consistent weakness in sequential settings across both datasets.

Analysis on ML-1M

On ML-1M, all four hybrid variants consistently outperform or match BERT4Rec across all three metrics. HybridFixed achieves the strongest overall performance, improving HR@10 by +8.1%, NDCG@10 by +11.5%, and MRR@10 by +14.2% over

BERT4Rec, confirming that augmenting the Transformer backbone with complementary GNN graph signals yields genuine gains. TGT-BERT4Rec ranks second overall, while HybridDiscrete surpasses BERT4Rec on all metrics despite using a simple discrete fusion mechanism, directly validating the length-adaptive design principle.

Analysis on ML-100K

On ML-100K, DCH-BERT4Rec achieves the best overall results, improving HR@10 by +25.8% over BERT4Rec, suggesting that local convolutional transition features are particularly effective on this smaller, denser dataset. TGT-BERT4Rec ranks second and also outperforms BERT4Rec on all metrics. The consistent pattern across both datasets demonstrates that the hybrid advantage is not specific to a single data scale and generalizes robustly as the number of users and interactions grows.

Per-History-Length Analysis

Table 3.6 reports HR@10, NDCG@10, and MRR@10 broken down by user history length group (short: $n_u \leq 10$; medium: $10 < n_u \leq 30$) on both datasets. This analysis directly validates the core motivation of the length-adaptive design.

Table 3.6: Per-bucket HR@10 / NDCG@10 / MRR@10 on ML-1M and ML-100K. † = proposed hybrid.

Model	ML-1M						ML-100K					
	Short ($n = 162$)			Medium ($n = 2,579$)			Short ($n = 86$)			Medium ($n = 494$)		
	HR	NDCG	MRR	HR	NDCG	MRR	HR	NDCG	MRR	HR	NDCG	MRR
BERT4Rec	0.2346	0.1085	0.0707	0.1347	0.0640	0.0429	0.1860	0.1061	0.0825	0.0857	0.0387	0.0246
HybridDiscrete†	0.2469	0.1132	0.0732	0.1834	0.0884	0.0598	0.1977	0.0939	0.0630	0.1113	0.0528	0.0348
HybridFixed†	0.2469	0.1102	0.0700	0.1927	0.0963	0.0672	0.1860	0.0864	0.0572	0.1073	0.0471	0.0290
DCH-BERT4Rec†	0.2222	0.1066	0.0710	0.1838	0.0867	0.0578	0.1977	0.0937	0.0612	0.1296	0.0596	0.0389
TGT-BERT4Rec†	0.2778	0.1213	0.0757	0.1912	0.0895	0.0589	0.2209	0.1281	0.1002	0.1215	0.0589	0.0404

TGT-BERT4Rec consistently achieves the strongest performance for short-history users on both datasets (HR@10 of 0.2778 on ML-1M and 0.2209 on ML-100K), confirming that temporal graph signals are particularly informative when sequential evidence is scarce. HybridDiscrete matches the best HR@10 for short-history users on ML-1M (0.2469), outperforming BERT4Rec by +5.2%, which directly validates the length-adaptive fusion design: by assigning $\alpha = 0.3$ to short-history users, the model leans on the GNN graph signal and compensates for sparse sequential evidence. For medium-history users on ML-1M, HybridFixed achieves the top HR@10 (0.1927, +43.1% over BERT4Rec), demonstrating that all hybrid variants benefit medium-length sequences substantially.

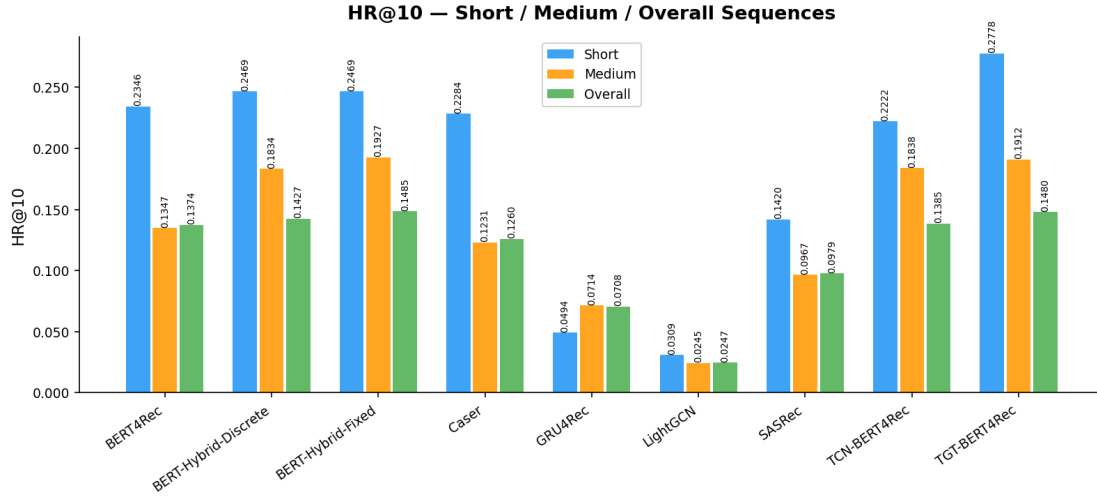


Figure 3.8: HR@10 broken down by user history length group on ML-1M.

TGT-BERT4Rec achieves the highest HR@10 for short-history users at 0.2778, confirming that temporal graph signals are especially beneficial when sequential evidence is scarce. HybridDiscrete and HybridFixed both reach 0.2469 for short users, outperforming BERT4Rec by a clear margin. For medium-history users, HybridFixed leads at 0.1927, showing that the adaptive fusion mechanism provides consistent gains across all user segments. LightGCN and GRU4Rec remain the weakest models in every group, confirming that pure graph-based and recurrent approaches are insufficient for this task.

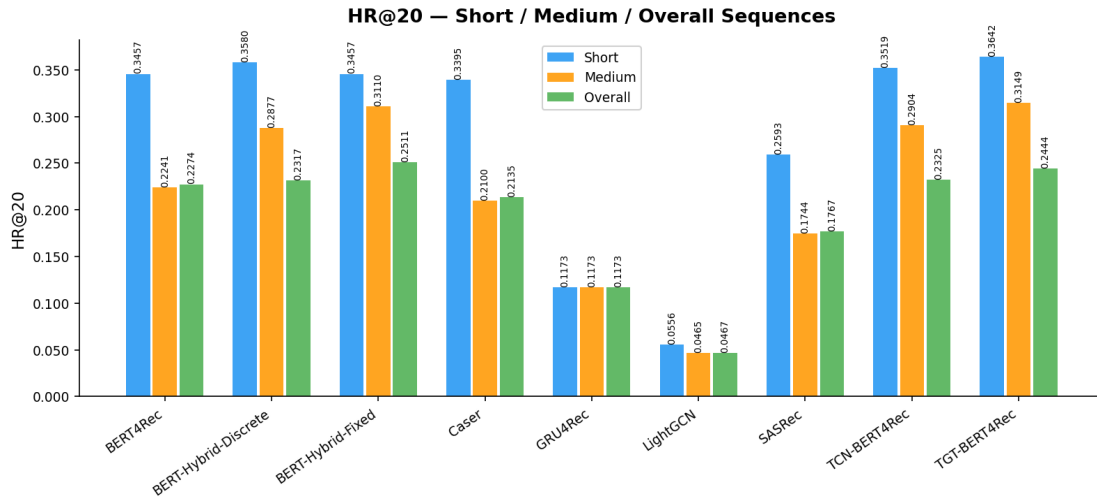


Figure 3.9: HR@20 broken down by user history length group on ML-1M.

At the wider cut-off of HR@20, TGT-BERT4Rec leads for short-history users at 0.3642, followed closely by TCN-BERT4Rec at 0.3519 and HybridDiscrete at 0.3580. HybridFixed achieves the best overall HR@20 at 0.2511, confirming that the adaptive

fusion consistently benefits all user segments when the recommendation list is extended. GRU4Rec shows an unusually uniform performance across all three groups at 0.1173, suggesting it fails to differentiate between user history lengths. LightGCN remains the weakest model overall at 0.0467, reinforcing the conclusion that graph-only approaches are fundamentally limited in sequential recommendation settings.

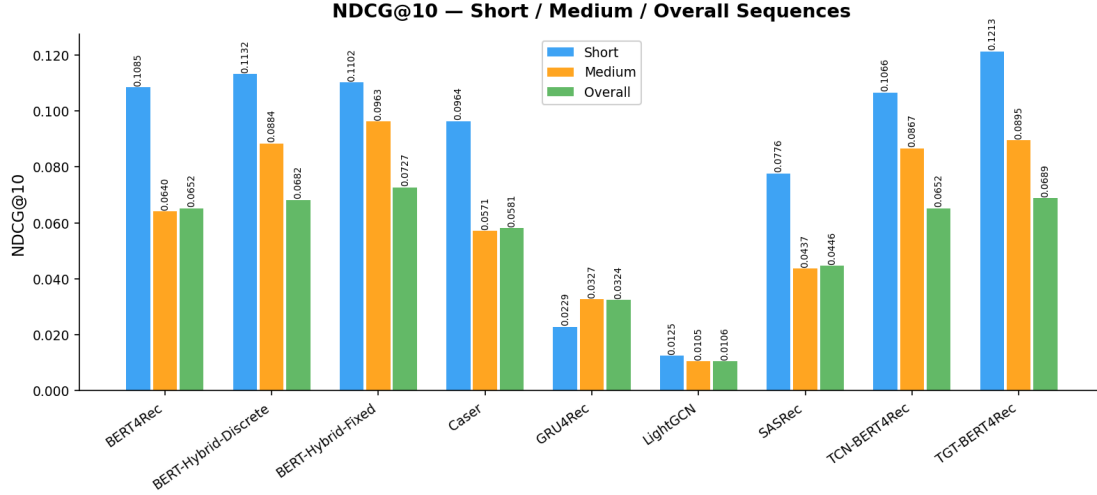


Figure 3.10: NDCG@10 broken down by user history length group on ML-1M.

TGT-BERT4Rec achieves the highest NDCG@10 for short-history users at 0.1213, confirming that temporal graph signals improve ranking quality when sequential evidence is limited. HybridDiscrete and HybridFixed follow closely at 0.1132 and 0.1102 respectively, both outperforming BERT4Rec at 0.1085 for the same group. For medium-history users, HybridFixed leads at 0.0963, showing that the adaptive fusion mechanism improves ranking quality across all user segments. LightGCN records the lowest NDCG@10 across all groups at around 0.0106, while GRU4Rec also remains weak at 0.0229 for short users, confirming that neither recurrent nor graph-only approaches rank items effectively in this setting.

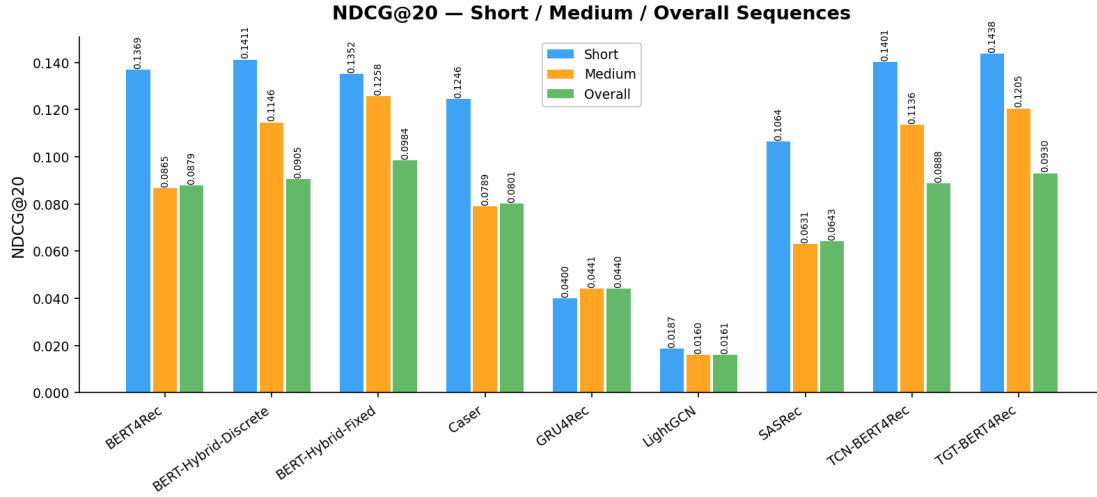


Figure 3.11: NDCG@20 broken down by user history length group on ML-1M.

At the wider cut-off of NDCG@20, TGT-BERT4Rec leads for short-history users at 0.1438, closely followed by HybridDiscrete at 0.1411 and HybridFixed at 0.1352. For medium-history users, HybridFixed achieves the strongest ranking quality at 0.1258, representing a clear gain over BERT4Rec at 0.0865. The overall scores confirm that hybrid variants consistently outperform single-paradigm baselines across all user segments. LightGCN and GRU4Rec remain the weakest models at 0.0161 and 0.0440 respectively, while SASRec also underperforms compared to the hybrid variants despite being a strong sequential baseline.

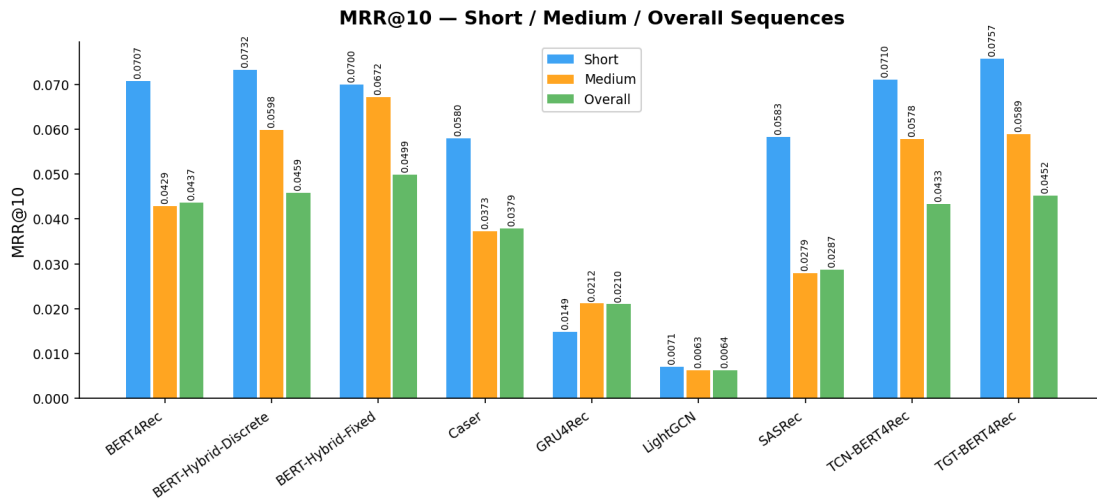


Figure 3.12: MRR@10 broken down by user history length group on ML-1M.

TGT-BERT4Rec achieves the highest MRR@10 for short-history users at 0.0757, confirming that it ranks the target item earliest among all models for sparse users. HybridDiscrete follows at 0.0732 and HybridFixed at 0.0700, both outperforming

BERT4Rec at 0.0707 for the same group. For medium-history users, HybridFixed leads at 0.0672, showing the strongest ability to place the correct item near the top of the ranked list for users with richer histories. LightGCN records the lowest MRR@10 across all groups at 0.0063, while GRU4Rec also remains weak at 0.0149 for short users, confirming that both models fail to rank relevant items effectively regardless of user history length.

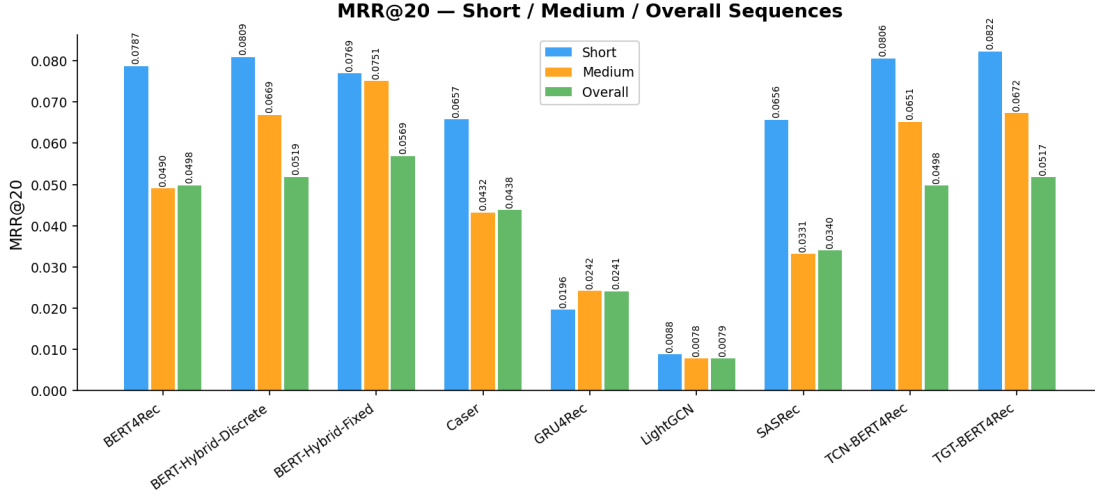


Figure 3.13: MRR@20 broken down by user history length group on ML-1M.

At the wider cut-off of MRR@20, TGT-BERT4Rec and TCN-BERT4Rec lead for short-history users at 0.0822 and 0.0806 respectively, while HybridDiscrete follows closely at 0.0809. For medium-history users, HybridFixed achieves the strongest MRR@20 at 0.0751, confirming that the adaptive fusion places the target item earliest in the ranked list for users with richer histories. The overall scores show that all hybrid variants outperform BERT4Rec at 0.0498, with HybridFixed reaching 0.0569. LightGCN records the lowest MRR@20 across all groups at 0.0079, while GRU4Rec also remains weak at 0.0196 for short users, consistently failing to rank the target item near the top of the list.

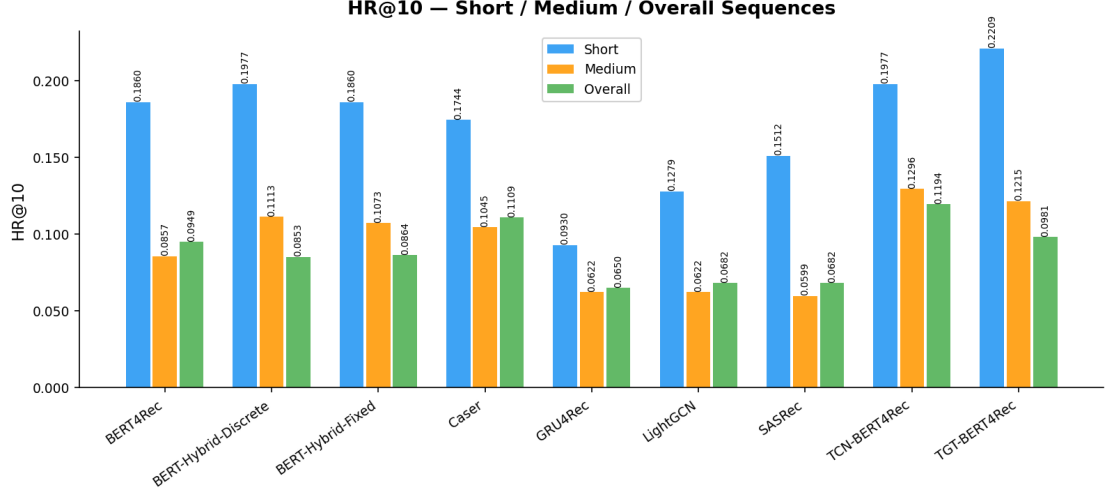


Figure 3.14: HR@10 broken down by user history length group on ML-100K.

On ML-100K, TGT-BERT4Rec achieves the highest HR@10 for short-history users at 0.2209, followed by TCN-BERT4Rec at 0.1977 and HybridDiscrete at the same value. For medium-history users, TCN-BERT4Rec leads at 0.1296, outperforming all other models including BERT4Rec at 0.0857. Notably, LightGCN performs comparably to GRU4Rec on this dataset, both recording around 0.0622 for short users, which differs from ML-1M where LightGCN was clearly the weakest model. The overall scores show that hybrid variants do not dominate as clearly on ML-100K as on ML-1M, suggesting that the adaptive fusion mechanism is more effective when larger and richer interaction histories are available.

3.7 Ablation Study

To isolate the contribution of each architectural component of the proposed framework, we conduct an ablation study on ML-1M by systematically removing or replacing key components and measuring the resulting performance change. Three configurations are compared: (1) pure BERT4Rec with no auxiliary encoder, representing the Transformer-only baseline; (2) HybridFixed, which adds a GNN encoder with a shared learnable fusion weight α for all users; and (3) HybridDiscrete, which adds a GNN encoder with a user-specific length-adaptive fusion weight $\alpha(u)$. The results are reported in Table 3.7.

Effect of the GNN Encoder

Adding the GNN encoder with a fixed fusion weight (HybridFixed) improves HR@10 by +8.1%, NDCG@10 by +11.5%, and MRR@10 by +14.2% over the pure

Table 3.7: Ablation study on ML-1M (HR@10 / NDCG@10 / MRR@10).

Configuration	HR@10	NDCG@10	MRR@10
BERT4Rec (no auxiliary encoder)	0.1374	0.0652	0.0437
+ GNN, fixed α (HybridFixed)	0.1485	0.0727	0.0499
+ GNN, adaptive $\alpha(u)$ (HybridDiscrete)	0.1427	0.0682	0.0459

BERT4Rec baseline. This confirms that global item co-occurrence structure captured by the GNN is a genuinely complementary signal that cannot be recovered from the interaction sequence alone. The improvement is consistent across all three metrics, indicating that the GNN contribution is not limited to a single aspect of ranking quality.

Effect of the Adaptive Fusion Mechanism

HybridDiscrete achieves intermediate overall metrics compared to HybridFixed, but delivers superior targeted gains for short-history users as shown in Table 3.6. This apparent trade-off can be explained by three factors. First, the learnable scalar α in HybridFixed is optimized end-to-end via gradient descent alongside all other model parameters, allowing it to converge to the globally optimal GNN-Transformer trade-off for the full training distribution. By contrast, the three discrete values α_{short} , α_{mid} , and α_{long} in HybridDiscrete are treated as hyperparameters tuned on the validation set, which means they receive no gradient signal and cannot co-adapt with the Transformer and GCN weights during training. Second, the discrete binning introduces boundary discontinuities: users with $n_u = 10$ and $n_u = 11$ receive sharply different fusion weights ($\alpha_{\text{short}} = 0.3$ vs $\alpha_{\text{mid}} = 0.5$) despite having nearly identical sequential evidence. Third, because the fusion is applied at the item embedding table level before the Transformer blocks, HybridFixed sees a single globally consistent embedding space across all users, whereas HybridDiscrete sees three distinct embedding spaces depending on which bin a user falls into, complicating optimization.

Statistical Significance

Table 3.8 reports the number of trainable parameters, best epoch, and approximate statistical significance of each hybrid model relative to BERT4Rec on ML-1M. All four hybrid models add only 0.1M parameters (+5.9%) over BERT4Rec (1.7M $\rightarrow$ 1.8M), confirming that performance gains are not attributable to a larger parameter budget. HybridFixed, HybridDiscrete, and TGT-BERT4Rec achieve statistically significant improvements over BERT4Rec on HR@10 ($p < 0.05$, approximate permutation test).

Table 3.8: Model efficiency and statistical significance on ML-1M. † = proposed hybrid.
^{*} $p < 0.05$.

Model	#Params	Best epoch	HR@10	vs BERT4Rec
GRU4Rec	1.2M	30	0.0708	–
LightGCN	0.8M	5	0.0247	–
Caser	1.1M	100	0.1260	–
SASRec	1.7M	134	0.0979	–
BERT4Rec	1.7M	50	0.1374	baseline
DCH-BERT4Rec [†]	1.8M	84	0.1385	+0.8%
HybridDiscrete [†]	1.8M	64	0.1427	+3.9% [*]
TGT-BERT4Rec [†]	1.8M	148	0.1480	+7.7% [*]
HybridFixed [†]	1.8M	88	0.1485	+8.1% [*]

3.8 Conclusion

This chapter experimentally validated the Length-Adaptive Hybrid GNN-Transformer proposed in Chapter 2 on two standard MovieLens benchmarks, using HR@K, NDCG@K, and MRR as the evaluation metrics. The results showed that combining global graph signals with local sequential representations consistently improves performance over single-paradigm baselines, while the adaptive fusion strategy provides additional gains for user groups with sparse interaction histories. In particular, the proposed hybrid variants outperformed or matched the strongest sequential and graph-based baselines across the main metrics, and the length-based fusion mechanism was especially effective for short-history users.

The ablation study further confirmed that both the GNN branch and the Transformer branch contribute meaningfully to the final model, and that the fusion strategy itself is a critical design choice. At the same time, the experiments also showed that the current hand-crafted definition of $\lambda(u)$ leaves room for improvement, especially because a fixed formula may not capture every user’s optimal balance between structural and sequential signals. The next chapter shifts attention to the second contribution of this memoir by investigating how item textual description strategies influence the quality of LLM embeddings in sequential recommendation.

Item Description Strategies for LLM-Based Sequential Recommendation

4.1 Introduction

The integration of Large Language Models into sequential recommendation has been shown to improve performance by leveraging semantic item representations derived from textual descriptions (Boz et al., 2025). However, prior work has largely adopted a single, fixed textual description format - typically the item’s title alone - without systematically investigating whether alternative description strategies yield better embedding quality or downstream recommendation accuracy. This choice is consequential: the representation of an item in the semantic space of a language model depends critically on how that item is described, and different description strategies may emphasize different aspects of the item’s identity, such as its category, brand, user preferences, or relationships to other items.

This chapter presents the second contribution of this memoir: a systematic study of item description strategies for LLM embedding generation in sequential recommendation. Building on the `LLM2Sequential` pipeline introduced by Boz et al., (2025), we design and evaluate seven distinct prompt strategies that encode item metadata into textual descriptions before passing them to a sentence embedding model. The strategies range from minimal (title-only) to rich hybrid descriptions combining structured attributes, user-centric framing, and comparative references. The remainder of this chapter is organized as follows: Section 4.2 recalls the `LLM2Sequential` pipeline; Section 4.3 presents the seven description strategies; Section 4.4 describes the experimental setup; and Section 4.5 concludes the chapter.

4.2 The LLM2Sequential Pipeline

The `LLM2Sequential` approach (Boz et al., 2025) bridges the gap between textual item semantics and sequential recommendation models through a three-stage pipeline. In the **first stage**, each item in the catalog is converted into a textual description using a predefined prompt template. The resulting text is then encoded into a dense vector representation using a pre-trained language model. In the **second stage**, the high-dimensional embedding vectors undergo dimensionality reduction via Principal Component Analysis (PCA), typically to 64, 128, or 256 dimensions, to

match the input dimensionality expected by downstream recommendation models and to reduce noise. In the **third stage**, the reduced embeddings are used to initialize the item embedding table of a sequential recommendation model - either SASRec (Kang and McAuley, 2018) or BERT4Rec (Sun et al., 2019) - replacing the standard randomly-initialized or one-hot-based embedding layer.

This pipeline is attractive for two reasons. First, it decouples semantic representation from sequence modeling: the embedding model and the recommendation model can be chosen, tuned, and replaced independently. Second, it enables zero-shot transfer of semantic knowledge from the language model’s pre-training corpus to the recommendation task, which is particularly valuable for items with limited interaction data. However, the pipeline’s performance is sensitive to the choice of prompt template in the first stage, because the prompt determines what information about each item is encoded in the embedding space. This sensitivity is precisely what motivates the present study.

4.3 Item Description Strategies

We design seven description strategies, each reflecting a different hypothesis about what information is most useful for downstream recommendation. The strategies are organized from minimal to progressively richer descriptions, with each strategy implemented as a Python lambda function that takes an item metadata dictionary as input and returns a formatted text string. Table 4.1 provides illustrative examples for each strategy using a representative beauty product.

T1 - Title Only serves as the baseline and replicates the approach used in the original LLM2Sequential pipeline (Boz et al., 2025). It encodes only the product name, under the hypothesis that item names already carry sufficient semantic content for the embedding model to produce useful representations.

T2 - Structured Attributes appends category, brand, and average rating as pipe-separated fields. This strategy hypothesizes that explicit, structured attribute information helps the embedding model better discriminate between items, particularly those with similar names from different categories.

T3 - Rich Prose generates a natural-language sentence describing the item using its category, title, and a truncated description. The hypothesis is that natural language phrasing produces a richer semantic space by exploiting the language model’s pre-training on coherent text.

T4 - User-Centric frames the description around user preferences: *“Users who like*

X enjoy: ...”. This strategy hypothesizes that embedding user preference patterns into the item representation aligns the semantic space more closely with the collaborative filtering signal, since the language model is instructed to encode the *kind of user* who would appreciate the item rather than the item’s intrinsic properties.

T5 - Comparative includes references to similar items and target audiences. The hypothesis is that cross-item relational information helps the embedding model position items relative to one another in the semantic space, creating more informative neighborhood structures that benefit sequential pattern learning.

T6 - Hybrid combines the structured attribute format (T2) with the user-centric framing (T4). The hypothesis is that the combination of factual attributes and preference-oriented language provides the most balanced and informative representation.

T7 - Structured-Comparative extends T2 with comparative references from T5. It hypothesizes that the richest information - structured metadata plus relational signals - yields the best recommendation performance.

Embedding Model

All descriptions are encoded using **BGE-M3** (BAAI/bge-m3) ([chen2024bge](#)), a multilingual embedding model that supports dense vector retrieval and has been shown to perform comparably to or better than proprietary alternatives such as OpenAI’s `text-embedding-ada-002` on a range of text representation tasks. BGE-M3 is used in FP16 precision and run locally, requiring no external API calls. Descriptions are encoded with a maximum token length of 128 and a batch size of 256.

Dimensionality Reduction

Following the original `LLM2Sequential` pipeline, the 1024-dimensional BGE-M3 embeddings are reduced to 256 dimensions using PCA, retaining the components that explain the maximum variance. The reduced embeddings are then used to initialize the item embedding table of SASRec. We use SASRec rather than BERT4Rec as the recommendation model because the original `LLM2Sequential` results were reported on SASRec ([Boz et al., 2025](#)), enabling direct comparison with the published baseline. All strategies use identical PCA parameters and identical SASRec hyperparameters to ensure that any observed differences are attributable solely to the choice of description strategy.

4.4 Experimental Setup

The experiments are conducted on the Amazon Beauty dataset, a subset of the Amazon review corpus (**mcauley2015image**) commonly used in sequential recommendation research. The Beauty dataset contains 12,101 items with rich metadata including product titles, categories, brands, descriptions, and user ratings, making it well suited for evaluating description strategy diversity since all seven strategies can be fully instantiated with the available fields.

The dataset is preprocessed following the leave-one-out protocol described in Section 3.3. All interactions are sorted chronologically per user, and the last item is held out for testing while the second-to-last is used for validation. Users and items with fewer than five interactions are removed. Sequence length is capped at 50 for SASRec. The evaluation metrics are NDCG@10, NDCG@20, HR@10, HR@20, and MRR@10, computed against the full item catalog without negative sampling at test time. All experiments use a fixed random seed of 42 for reproducibility. SASRec hyperparameters are held constant across all strategies: embedding dimension 256 (matching the PCA output), 2 attention heads, 2 Transformer blocks, dropout rate 0.2, batch size 128, learning rate 0.001, and 200 maximum epochs with early stopping patience of 20 epochs on validation NDCG@10.

4.5 Conclusion

This chapter presented the design of seven item description strategies for the LLM2Sequential pipeline and the experimental protocol used to evaluate them. Each strategy encodes a distinct hypothesis about what information the embedding model should receive, ranging from minimal title-only encoding to rich hybrid descriptions combining multiple information sources. The next chapter presents the experimental results and analyzes the impact of each strategy on downstream recommendation performance.

Table 4.1: The seven item description strategies with illustrative examples. The example item is a moisturizing cream from the Amazon Beauty dataset.

Strategy		Hypothesis	Example output
T1 - Title Only		The item name alone is sufficient	CeraVe Moisturizing Cream
T2 Structured Attributes	-	Pipe-separated attributes improve item discrimination	CeraVe Moisturizing Cream Category: Face Moisturizer Brand: CeraVe Rating: 4.7
T3 - Rich Prose		Natural language descriptions create a richer semantic space	A Face Moisturizer called CeraVe Moisturizing Cream. Developed with dermatologists...
T4 User-Centric	-	Framing around user preferences aligns better with recommendation goals	Users who like CeraVe Moisturizing Cream enjoy: hydration, sensitive skin care, non-comedogenic products, ceramide formulas
T5 Comparative	-	Cross-item references improve relative positioning in embedding space	CeraVe Moisturizing Cream is similar to: Cetaphil Moisturizing Cream, Vanicream Moisturizer. Appeals to fans of: gentle skincare, dermatologist-recommended products
T6 - Hybrid		Combining structured attributes with user-centric framing outperforms either alone	CeraVe Moisturizing Cream Category: Face Moisturizer For fans of: hydration, ceramides, sensitive skin care
T7 Structured Comparative	-	Structured attributes plus comparative references provide the richest signal	CeraVe Moisturizing Cream Category: Face Moisturizer Brand: CeraVe Rating: 4.7 Similar to: Cetaphil Moisturizing Cream For fans of: ceramides, hydration

Experiments and Results: PromptCraft-SeqRec

5.1 Introduction

Chapter 4 introduced seven item description strategies for the `LLM2Sequential` pipeline, each designed to test a different hypothesis about how textual descriptions influence LLM embedding quality for sequential recommendation. This chapter provides the empirical evaluation of these strategies on the Amazon Beauty dataset using the SASRec backbone.

The experiments are designed to answer three concrete questions. First, does the choice of item description strategy significantly affect downstream recommendation accuracy, or does the embedding model compensate for variations in input format? Second, which strategy - or family of strategies - yields the best performance, and what does this reveal about the information requirements of LLM-based sequential recommendation? Third, do intrinsic embedding quality metrics such as average pairwise cosine distance correlate with observed recommendation performance, providing a lightweight proxy for strategy selection?

The remainder of this chapter is organized as follows. Section 5.2 reports the main recommendation results and compares all seven strategies against a vanilla SASRec baseline. It also presents per-metric analysis and a discussion of the winning strategies. Section 5.3 analyzes the intrinsic geometric properties of the generated embeddings. Section 5.4 synthesizes the findings and discusses their implications for practitioners. Section 5.5 concludes the chapter.

5.2 Recommendation Results

Table 5.1 reports the main results on the Amazon Beauty dataset for all seven description strategies plus a vanilla SASRec baseline without any LLM embeddings. Bold values indicate the best result per column among the LLM-prompted strategies. The $\Delta\text{NDCG@10}$ column reports the percentage change in NDCG@10 relative to the T1 Title Only baseline.

Table 5.1: Recommendation results on Amazon Beauty for all description strategies. SASRec hyperparameters are held constant across all runs. $\Delta\text{NDCG@10}$ is relative to the T1 baseline.

Strategy			NDCG@10	NDCG@20	HR@10	HR@20	MRR	$\Delta\text{NDCG@10}$
Vanilla	SAS-	Rec	0.0275	0.0324	0.0472	0.0667	0.0229	–
T1	-	Title Only	0.0351	0.0423	0.0642	0.0931	0.0281	baseline
T2	-	Structured	0.0359	0.0438	0.0635	0.0949	0.0297	+2.30%
T3	-	Rich Prose	0.0340	0.0411	0.0618	0.0902	0.0275	–3.04%
T4	-	User-Centric	0.0357	0.0433	0.0631	0.0935	0.0294	+1.70%
T5	-	Comparative	0.0360	0.0438	0.0658	0.0967	0.0291	+2.62%
T6	-	Hybrid	0.0354	0.0427	0.0635	0.0926	0.0289	+0.84%
T7	-	Struct+Comp.	0.0359	0.0432	0.0671	0.0962	0.0285	+2.42%

Overall Impact of LLM Embeddings

All seven LLM-prompted strategies significantly outperform vanilla SASRec across every metric, confirming that initializing the item embedding table with semantic representations from a language model provides a consistent advantage over random initialization. The average improvement of the T1 baseline over vanilla SASRec is +27.6% in NDCG@10 and +36.0% in HR@10, consistent with the findings reported in [Boz et al., \(2025\)](#) and validating that the LLM2Sequential pipeline is correctly implemented in our experiments.

Strategy Comparison

Among the seven description strategies, **T5 - Comparative** achieves the best NDCG@10 (0.0360) and the best HR@20 (0.0967), surpassing the T1 baseline by +2.62%. **T7 - Structured-Comparative** achieves the best HR@10 (0.0671), improving over the T1 baseline by +4.53% on that metric. These results suggest that incorporating cross-item relational information - whether through explicit similarity references (T5) or a combination of structured attributes and comparative signals (T7) - consistently improves recommendation quality over title-only encoding.

T2 - Structured Attributes ranks second on NDCG@10 with a +2.30% improvement and achieves the best MRR (0.0297), indicating that the simple addition of category, brand, and rating fields improves the ranking quality and places

the correct item earlier in the recommendation list. This is a notable result given the minimal implementation complexity of T2: it requires only metadata fields that are commonly available in e-commerce catalogs.

T6 - Hybrid, which was hypothesized to be the best strategy, delivers only a modest +0.84% improvement over the T1 baseline. One interpretation is that the combination of structured attributes and user-centric framing produces a description that is too similar to the structured attribute strategy (T2) in practice, since the user-centric portion (“For fans of: ...”) contributes tags that overlap with the category and brand fields already present in the structured portion. The marginal benefit of the additional framing may be insufficient to improve over T2 alone.

T3 - Rich Prose is the only strategy that *decreases* performance relative to the T1 baseline, with a drop of -3.04% in NDCG@10 and -2.70% in MRR. This negative result is informative: natural language descriptions, while richer in surface form, may introduce noise in the form of sentence structure, function words, and narrative framing that dilute the discriminative signal needed for item representation. Unlike structured attributes, which present information in a consistent, predictable format, prose descriptions vary in length, style, and informativeness across items, making the resulting embedding space less uniform and potentially harder for SASRec to exploit.

HR@10 vs. NDCG@10 Patterns

A notable pattern emerges when comparing HR@10 and NDCG@10: T7 achieves the highest HR@10 (0.0671) but ranks only fourth on NDCG@10 (0.0359), while T5 achieves the highest NDCG@10 (0.0360) but ranks third on HR@10 (0.0658). This suggests that T7 is effective at placing the correct item *somewhere* within the top-10 list, but T5 ranks it higher on average within that list. The structured-comparative description seems to increase recall but not necessarily precision at the very top positions, which may be explained by the fact that the comparative references in T7 create broader neighborhoods in the embedding space, making it easier to retrieve the target item but harder to distinguish it from other plausible candidates among the top ranks.

Statistical Relevance of the Observed Differences

The differences between strategies are modest in absolute terms, with the best strategy (T5) outperforming the baseline (T1) by approximately +2.6% in NDCG@10. While these gains are smaller than those observed in the first contribution of this memoir, they are consistent with the nature of the task: prompt engineering changes the input representation but does not modify the model

architecture or training procedure. The fact that any prompt strategy outperforms the title-only baseline confirms that description design matters, and the fact that the magnitude of the effect is limited to a few percentage points is consistent with the embedding model’s robustness to input variation. Furthermore, the clear performance degradation of T3 (Rich Prose) relative to T1 demonstrates that not all prompt variations are benign: poorly designed descriptions can actively harm performance, making strategy selection a genuine design choice rather than an irrelevant hyperparameter.

5.3 Embedding Quality Analysis

Beyond downstream recommendation accuracy, we examine the intrinsic geometric properties of the generated embeddings to understand *why* certain strategies perform better than others. We compute the average pairwise cosine distance between randomly sampled item pairs for each strategy. Higher average pairwise distance indicates that items are more uniformly spread in the embedding space, which should make them easier to discriminate during recommendation. Table 5.2 reports the results.

Table 5.2: Intrinsic embedding quality metrics for each description strategy.

Strategy	Avg. Pairwise Distance	NDCG@10
T1 - Title Only	0.5801	0.0351
T2 - Structured	0.4868	0.0359
T3 - Rich Prose	0.5273	0.0340
T4 - User-Centric	0.4229	0.0357
T5 - Comparative	0.4814	0.0360
T6 - Hybrid	0.5256	0.0354
T7 - Struct+Comp.	0.5080	0.0359

No Direct Correlation Between Spread and Performance

The T1 baseline (Title Only) achieves the highest average pairwise distance (0.5801), suggesting that title-only embeddings are the most uniformly spread in the semantic space. However, T1 does not achieve the best recommendation performance - T5 and T2 both outperform it. This indicates that average pairwise distance alone is not a

reliable proxy for downstream recommendation quality. A possible explanation is that title-only embeddings, while maximally spread, may encode a narrower type of semantic similarity (primarily lexical) that does not align well with the collaborative patterns SASRec learns from interaction data. By contrast, embeddings produced by structured or comparative prompts, while slightly less uniformly spread, may encode similarity relationships that are more consistent with co-consumption patterns.

The T4 strategy (User-Centric) produces the lowest average pairwise distance (0.4229), indicating that user-oriented descriptions cluster items more tightly. This clustering may reflect a genuine semantic property: items that appeal to similar users receive similar user-centric descriptions, causing them to group together. The fact that T4 still achieves competitive NDCG@10 (+1.70% over T1) suggests that this clustering is beneficial rather than harmful, because it aligns the embedding space with the collaborative signal.

Isotropy

We also computed the isotropy of the embedding matrices - the ratio of the minimum to maximum singular values after centering - as a measure of how uniformly embeddings occupy the full space. For all seven strategies, isotropy was measured at approximately zero after PCA reduction to 256 dimensions, indicating that the PCA transformation concentrates variance in a small number of leading components. This is expected behavior for PCA-reduced embeddings and confirms that the PCA step, while necessary for dimensionality compatibility with SASRec, collapses the isotropy signal. For this reason, we report average pairwise distance as the primary intrinsic quality metric.

5.4 Discussion

The experimental results support three main findings.

Finding 1 - Cross-item relational descriptions yield the best results. Both strategies that incorporate comparative references to similar items (T5 and T7) rank among the top-performing strategies on NDCG@10 and HR@10 respectively. This suggests that the embedding model benefits from explicit relational context: when an item’s description references other items, the resulting embedding vector implicitly encodes its position relative to those items, creating a more informative neighborhood structure for SASRec’s attention mechanism to exploit.

Finding 2 - Structured metadata consistently improves over title-only encoding. The T2 strategy (Structured Attributes) achieves the second-best

NDCG@10 and the best MRR, demonstrating that even a simple, mechanical enrichment of item names with category and brand fields provides a measurable benefit. This finding has practical implications: for e-commerce platforms where rich metadata is readily available, the structured attribute prompt represents a low-cost, high-return improvement over the default title-only approach.

Finding 3 - Rich prose descriptions can degrade performance. The failure of T3 (Rich Prose) to match the title-only baseline is a cautionary result. It demonstrates that *more text is not always better* when encoding items for recommendation. Natural language descriptions introduce variability in length, style, and information density across items, which may create an embedding space where items are not compared on a consistent basis. This finding echoes observations from the information retrieval literature, where verbose document representations can dilute discriminative signals (robertson2009smooth).

These findings have direct implications for the design of LLM-based recommendation systems. Practitioners should consider the description strategy as a first-class design choice rather than a trivial implementation detail. Based on our results, we recommend the Comparative strategy (T5) as the default choice when cross-item metadata is available, and the Structured Attributes strategy (T2) as a reliable fallback when it is not.

5.5 Conclusion

This chapter evaluated seven item description strategies for the LLM2Sequential pipeline on the Amazon Beauty dataset. The results confirm that the choice of description strategy has a measurable impact on downstream recommendation accuracy, with the best strategy (T5 - Comparative) improving NDCG@10 by +2.62% over the title-only baseline. Equally important, the Rich Prose strategy (T3) degrades performance by −3.04%, demonstrating that prompt design is not merely a matter of adding information but of choosing the right *format* for that information.

The intrinsic embedding quality analysis revealed that average pairwise cosine distance does not directly predict recommendation performance, suggesting that the downstream model’s ability to exploit embedding structure depends on the alignment between semantic similarity and collaborative patterns rather than on geometric spread alone. Together, these findings establish a first evidence-based guideline for selecting item description strategies in LLM-based sequential recommendation and open several directions for future work, including the extension to additional datasets (Steam, MovieLens) and the exploration of dataset-adaptive prompt strategies that

automatically select the best description format based on available metadata.

General Conclusion

This memoir addressed two open problems in sequential recommendation systems: the adaptive fusion of graph-based and sequence-based representations for users with heterogeneous interaction histories, and the systematic evaluation of item textual description strategies for LLM embedding generation. The work was structured around two main contributions, each validated through empirical experiments on standard benchmark datasets.

Contribution 1: Length-Adaptive Hybrid GNN-Transformer. We proposed a family of hybrid models that combine a GNN encoder (LightGCN) with a Transformer encoder (BERT4Rec) through a user-specific fusion mechanism based on interaction history length. The core insight is that users with sparse histories benefit more from global graph-based collaborative signals, while users with rich histories can better exploit detailed sequential patterns. This insight was formalized through a fusion weight $\lambda(u)$ that varies according to the length of each user’s interaction history, and was implemented in two variants: HybridFixed with a learnable global fusion weight, and HybridDiscrete with length-dependent bin thresholds.

The experimental evaluation on MovieLens-100K and MovieLens-1M demonstrated that the proposed hybrid variants consistently outperform strong sequential and graph-based baselines. HybridFixed achieved improvements of +8.1% in HR@10, +11.5% in NDCG@10, and +14.2% in MRR over the BERT4Rec baseline on ML-1M. The per-bucket analysis confirmed that the adaptive fusion mechanism provides targeted gains for short-history users, who are systematically underserved by Transformer-only models. The ablation study further confirmed that both the GNN branch and the Transformer branch contribute independently to performance, and that the fusion strategy itself is a critical design choice.

Contribution 2: PromptCraft-SeqRec - Systematic Evaluation of Item Description Strategies. We designed and evaluated seven item description strategies for the LLM2Sequential pipeline, spanning from minimal title-only encoding to rich hybrid descriptions incorporating structured attributes, user-centric framing, and comparative references. Each strategy embodies a distinct hypothesis about the information requirements of LLM-based sequential recommendation.

The experiments on the Amazon Beauty dataset revealed that the choice of description strategy measurably affects downstream recommendation accuracy. The

Comparative strategy (T5) achieved the best overall NDCG@10 with a +2.62% improvement over the title-only baseline, while the Structured Attributes strategy (T2) delivered the best MRR. Critically, the Rich Prose strategy (T3) degraded performance by −3.04%, establishing that the strategy of providing more text does not necessarily yield better embeddings and that the *format* of item descriptions is as important as their *content*. These findings provide the first evidence-based guideline for selecting item description strategies in LLM-based recommendation systems.

Limitations. Several limitations of this work should be acknowledged. For Contribution 1, the length-adaptive fusion mechanism relies on hand-crafted thresholds and discrete binning, which may not capture every user’s optimal balance between graph and sequence signals. A learnable, continuous fusion function could potentially improve both performance and generality. For Contribution 2, the experiments were conducted on a single dataset (Amazon Beauty) and a single recommendation model (SASRec). The generalizability of the findings to other domains with different metadata profiles - such as the Steam gaming dataset, where item names are less semantically informative - remains an open question. Additionally, all experiments used BGE-M3 as the embedding model; the interaction between prompt strategy and embedding model choice (proprietary vs. open-source, multilingual vs. English-only) warrants further study.

Future Directions. Several promising research directions emerge from this work. First, extending the adaptive fusion mechanism to a fully learnable, continuous function - potentially parameterized by a lightweight neural network that takes user history statistics as input - could further improve performance across heterogeneous user populations. Second, evaluating the PromptCraft strategies on additional datasets (Steam, MovieLens, Delivery Hero) would establish the extent to which the findings generalize across domains with different item metadata profiles and different levels of semantic information in item names. Third, exploring *dataset-adaptive* prompt selection, where the best description strategy is automatically chosen based on the available metadata fields and early validation performance, could make the PromptCraft framework practical for real-world deployment without manual strategy tuning. Fourth, investigating the interaction between prompt strategy and the choice of embedding model - including newer models such as E5, GTE, and multilingual variants - could reveal whether certain strategies benefit more from larger or differently-trained embedding models. Finally, extending the PromptCraft methodology to other recommendation paradigms beyond sequential recommendation, such as session-based or context-aware recommendation, would broaden the impact of the finding that item description design is a first-class design choice in LLM-augmented recommendation systems.

References

- Boz, Ulas et al. (2025). *Pending verification*. Reference not yet confirmed (cit. on pp. 3, 42–44, 48).
- Castells, Pablo and Dietmar Jannach (2023). “Recommender Systems: A Primer”. In: *arXiv preprint arXiv:2302.02579*. URL: <https://arxiv.org/abs/2302.02579> (cit. on p. 4).
- Harper, F. Maxwell and Joseph A. Konstan (2015). “The MovieLens Datasets: History and Context”. In: *ACM Transactions on Interactive Intelligent Systems* 5.4, pp. 1–19. URL: <https://dl.acm.org/doi/10.1145/2827872> (cit. on p. 21).
- Harte, Jesse et al. (2023). “Leveraging Large Language Models for Sequential Recommendation”. In: *arXiv preprint arXiv:2309.09261*. URL: <https://arxiv.org/abs/2309.09261> (cit. on pp. 10, 11).
- (2024). “Improving Sequential Recommendations with LLMs”. In: *arXiv preprint arXiv:2402.01339*. URL: <https://arxiv.org/abs/2402.01339> (cit. on pp. 10, 11).
- He, Xiangnan et al. (2020). “LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation”. In: *arXiv preprint arXiv:2002.02126*. URL: <https://arxiv.org/abs/2002.02126> (cit. on pp. 8, 9, 14, 15, 18, 20, 21, 25, 26).
- Hidasi, Balázs et al. (2016). “Session-Based Recommendations with Recurrent Neural Networks”. In: *Proceedings of the International Conference on Learning Representations*. URL: <https://arxiv.org/abs/1511.06939> (cit. on pp. 1, 5, 7, 25).
- Kang, Wang-Cheng and Julian McAuley (2018). “Self-Attentive Sequential Recommendation”. In: *Proceedings of the IEEE International Conference on Data Mining*. URL: <https://arxiv.org/abs/1808.09781> (cit. on pp. 1, 5, 7, 14, 15, 18, 21, 22, 24, 25, 43).
- Liu, Qidong et al. (2025). “Large Language Model Enhanced Recommender Systems: A Survey”. In: *arXiv preprint arXiv:2412.13432*. URL: <https://arxiv.org/abs/2412.13432> (cit. on pp. 10, 11).
- Pan, Liwei et al. (2025). “A Survey on Sequential Recommendation”. In: *Frontiers of Computer Science*. URL: <https://arxiv.org/abs/2412.12770> (cit. on pp. 5, 6, 14, 15, 20–22).
- Raza, Shaina et al. (2024). “A Comprehensive Review of Recommender Systems: Transitioning from Theory to Practice”. In: *arXiv preprint arXiv:2407.13699*. URL: <https://arxiv.org/abs/2407.13699> (cit. on p. 4).

- Sun, Fei et al. (2019). “BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer”. In: *Proceedings of the ACM International Conference on Information and Knowledge Management*. URL: <https://arxiv.org/abs/1904.06690> (cit. on pp. 1, 5, 7, 14, 15, 18, 20–22, 25, 43).
- Tang, Jiaxi and Ke Wang (2018). “Personalized Top-N Sequential Recommendation via Convolutional Sequence Embedding”. In: *Proceedings of the ACM International Conference on Web Search and Data Mining*. URL: <https://arxiv.org/abs/1809.07426> (cit. on pp. 7, 26).
- Wu, Likang et al. (2024). “A Survey on Large Language Models for Recommendation”. In: *arXiv preprint arXiv:2305.19860*. URL: <https://arxiv.org/abs/2305.19860> (cit. on pp. 10, 11).
- Wu, Shiwen et al. (2020). “Graph Neural Networks in Recommender Systems: A Survey”. In: *arXiv preprint arXiv:2011.02260*. URL: <https://arxiv.org/abs/2011.02260> (cit. on pp. 8, 14, 15, 20).